

Python Programming for Beginners

Python Programming for Beginners

Prof. Rahul E. Borate

B.Sc., MCA

Dr. Sunil Khilari

Ph.D., MCA, MBA(IT), MCP, DSQM, ITIL, IPPro

Prof. Rahul S. Navale

B.Sc., MCA

**THE
X
TEAM**



SHROFF PUBLISHERS & DISTRIBUTORS PVT. LTD.

Mumbai

Bangalore

Kolkata

New Delhi

Python Programming for Beginners

By Prof. Rahul E. Borate, Dr. Sunil Khilari, Prof. Rahul S. Navale

Copyright © 2019, By Prof. Rahul E. Borate, Dr. Sunil Khilari, Prof. Rahul S. Navale

First Edition: July 2019

ISBN: 978-93-5213-000-0

All rights reserved. No part of the material protected by this copyright notice may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system, nor exported, without the written permission of the copyright owner or the publisher.

Published by **Shroff Publishers & Distributors Pvt. Ltd.** B-103, Railway Commercial Complex, Sector 3, Sanpada (E), Navi Mumbai 400705 • TEL: (91 22) 4158 4158 • FAX: (91 22) 4158 4141 • E-mail: spdorders@shroffpublishers.com • Web: www.shroffpublishers.com Printed at Jasmine Art Printers Pvt. Ltd., Navi Mumbai.

Dedication

Our Mother and Father

Preface

We have confidence that once you complete reading this book, you should gain an in-depth understanding about *Python*'s powerful multiparadigm concept, optimized for programmer productivity, code readability and software quality.

This book provides a comprehensive and in-depth introduction to the Python language. Its goal is to help you master Python fundamentals before moving on to applying them in your work. This book is designed to serve as a single, all-inclusive learning resource for all Python newcomers, whether they will be using Python 2.X, Python 3.X, or both.

Whether you are new to programming or are a professional developer, this book is designed to bring you up to speed on the Python language in ways that more limited approaches cannot. After reading this book, you should know enough about Python to apply it in whatever application domains you choose to explore.

What This Book Is Not

So now that we've told you what this book is, we also want to be clear on what it isn't:

This book is a tutorial, *not* a reference.

This book covers the language itself, *not* applications, standard libraries, or third-party tools.

This book is a comprehensive look at a substantial topic, *not* a watered-down overview.

Prof. Rahul Borate

Prof. Sunil Khilari

Prof. Rahul Navale

Acknowledgments

The book you are looking at represents the efforts of many Software Professionals around us; all their contributions have been useful. Material is sourced from various reference journals, websites, books, conferences, interviews and observations. We would like to extend thanks to those who helped us to make this book possible.

We acknowledge in particular, the encouragement from **Hon. Prof. M. N. Navale, the Founder President of Sinhgad Technical Education Society, Pune.**

We would like to **thank Dr. Chandrani Singh, Director Sinhgad Institute of Management, Vadgaon (bk), Pune.** and **Prof. Sunil Shastri sir** who inspired us to write a book on Python Language –“Python Programming for Beginners”.

Writing a book is a very involved process and there were many that played a key part in the book's success. We would like to thank all the members of the team at Shroff Publishing for making this possible.

Thanks to our family and colleagues for sharing their subject expertise on Python Programming Language experience and continued support to make this book more useful and outstanding.

Finally, we undoubtedly owe the greatest thanks to those who read this book.

Table of Content

CHAPTER 1: INTRODUCTION TO PYTHON	1
1.1 Getting Started: Introduction to Python- an interpreted high level language, interactive mode and script mode	2
1.2 Variables and Types-mutable and Immutable variable and Keywords	6
1.3 Operators and Operands in Python	7
1.4 Operator precedence, Expressions and Statements (Assignment statement)	11
1.5 Taking input (using raw_input() and input()) and displaying output - print statement	12
1.6 Comments in Python	13
Exercise	14
CHAPTER 2: CONDITIONAL AND LOOPING CONSTRUCT	15
2.1 if - else statement and nested if – else while, for, use of range function in for, Nested loops	16
2.2 break, continue, pass statement	24
2.3 Use of Compound Expression in conditional construct	26
2.4 Built-In Function, invoking built in functions	27
2.5 Functions from Math, Random, Time & Date Module	29
2.6 Module (Importing entire module or selected objects using from statement)	33
2.7 Composition	33
2.8 User Defined Function: Defining, invoking functions, passing parameters (default parameter values, keyword arguments)	34
2.9 Scope of Variables, Void Functions and Functions Returning Values Scope of Variables	38
Exercise	43

CHAPTER 3: STRINGS 45

3.1 Creating, Initializing and Accessing the Elements	46
3.2 String Operators	48
3.3 String built in functions & methods:	51
Exercise	67

CHAPTER 4: STRUCTURED PROGRAMMING: ITERATION CONTROL FLOW 69

4.1 Concept of Mutable Lists, Creating, Initializing and Accessing the Elements of List	70
4.2 List Operations (Concatenation, Repetition, Membership, List Slices), List Comprehensions	72
4.3 List Functions & Methods: append, extend, sort, remove, reverse, pop	74
4.4 Immutable concept, creating, initializing and accessing the elements in a tuple	78
4.5 Tuple Functions: cmp(), len(), max(), min(), tuple()	79
4.6 Concept of Sets, Creating, Initializing and Accessing the Elements of Sets	82
4.7 Sets Operation (Membership, Union, Intersection, Difference, and Symmetric Difference)	83
4.8 Concept of Key-Value Pair, Creating, Initializing and Accessing the Elements in a Dictionary	85
4.9 Traversing, Appending, Updating and Deleting Elements	87
4.10 Dictionary Functions & Methods: cmp, len, clear(), has_key(), items(), keys(), update(), values()	88
Exercise	92

CHAPTER 5: MODULES 93

5.1 Concept of Module: Executing Modules as Scripts, the Module Search Path, “Compiled” Python Files, Standard Modules: What is Module?	94
---	----

5.2 The dir() Function	97
5.3 Package	98
Exercise	101
CHAPTER 6: I/O AND FILE HANDLING	103
6.1 Output Formatting:	104
6.2 Filenames and Paths:.....	106
6.3 Reading and Writing Files:.....	109
Exercise:.....	113
CHAPTER 7: ERRORS AND EXCEPTIONS	115
7.1 Syntax Errors, Exceptions:.....	116
7.2 Handling Exceptions:.....	118
7.3 Python Exception(Except with No Exception) Example:	120
7.4 Raise an Exception:.....	123
7.5 User-defined Exceptions:.....	124
7.6 Clean-Up Actions (Try ... Finally):.....	125
Exercise:.....	128
CHAPTER 8: INTRODUCTION TO OBJECT ORIENTED CONCEPTS IN PYTHON	129
8.1 Object Oriented concepts	130
8.3 Classes, Class Objects, Instance Objects, Method Objects, Class and Instance Variables: Class and Instance Variables in Python	131
8.4 Inheritance	134
Exercise	137

Chapter 1

INTRODUCTION TO PYTHON

- ☑ *Getting Started: Introduction to Python- an interpreted high level language, interactive mode and script mode.*
- ☑ *Variables and Types-mutable and Immutable variable and Keywords.*
- ☑ *Operators and Operands in Python.*
- ☑ *Operator precedence, Expressions and Statements (Assignment statement)*
- ☑ *Taking input (using `raw_input()` and `input()`) and displaying output - print statement*
- ☑ *Comments in Python.*
- ☑ *Exercise 1*

1.1 Getting Started: Introduction to Python- an interpreted high level language, interactive mode and script mode:

Python History:

Python laid its foundation in the late 1980s.

The implementation of Python was started in the December 1989 by **Guido Van Rossum in Netherlands**.

Python is influenced by programming languages like:

ABC language.

Modula-3

Python is used for software development at companies and organizations such as Google, Yahoo, CERN, Industrial Light and Magic, and NASA.

What is Python?

Python is an object-oriented, high level language, interpreted, dynamic and multipurpose programming language.

Python is not intended to work on special area such as web programming. That is why it is known as *multipurpose* because it can be used with web, enterprise, 3D CAD etc.

We don't need to use data types to declare variable because it is *dynamically typed* so we can write `a=10` to declare an integer value in a variable.

Python makes the development and debugging *fast* because there is no compilation step included in python development.

Python Features

There are a lot of features provided by python programming language.

1) Easy to Use:

Python is easy to use and **high level language**. Thus it is programmer friendly language.

2) Interpreted Language:

Python is an **interpreted language** i.e. interpreter executes the code line by line at a time. This makes debugging easy and thus suitable for beginners.

3) Cross-platform language:

Python can run equally on different platforms such as **Windows, Linux, Unix, Macintosh** etc. Thus, Python is a portable language.

4) Free and Open Source:

Python language is freely available (**www.python.org**).The source-code is also available. Therefore it is open source.

5) Object-Oriented language:

Python supports object oriented language. Concept of classes and objects comes into existence.

6) Large Standard Library:

Python has a large and broad library.

7) GUI Programming:

Graphical user interfaces can be developed using Python.

Language comparison:

		Tcl	Perl	Python	JavaScript	Visual Basic
Speed	development	✓	✓	✓	✓	✓
	regexp	✓	✓	✓		
Breadth	Extensible	✓		✓		✓
	embeddable	✓		✓		
	easy GUI	✓		✓		✓
	net/web	✓	✓	✓	✓	✓
Enterprise	cross-platform	✓	✓	✓	✓	
	thread-safe	✓		✓		✓
	database access	✓	✓	✓	✓	✓

Python Applications:

Python as a whole can be used in any sphere of development. Let us see what are the major regions where Python proves to be handy.

4 Python Programming for Beginners

1) Console Based Application

Python can be used to develop console based applications.

For example: **IPython**.

2) Audio or Video based Applications

Python proves handy in multimedia section.

Some of real applications are: **TimPlayer, cplay** etc.

3) 3D CAD Applications

Fandango is a real application which provides full features of CAD.

4) Web Applications

Python can also be used to develop web based application. Some important developments are: **PythonWikiEngines, PythonBlogSoftware** etc.

5) Enterprise Applications

Python can be used to create applications which can be used within an Enterprise or an Organization.

Some real time applications are: **OpenErp, Tryton** etc.

6) Applications for Images

Using Python several applications can be developed for image.

Applications developed are: **VPython, imgSeek** etc.

There are several such applications which can be developed using Python

Python Installation:

1. To install Python, firstly download the Python distribution from www.python.org/ download.
2. Having downloaded the Python distribution now execute it

❖ SETTING PATH IN PYTHON

Before starting working with Python, a specific path is to set

To set path follow the steps:

Right click on My Computer ->Properties ->Advanced System setting
->Environment Variable ->New

In Variable name write path and in Variable value copy path up to C://
Python(i.e., path where Python is installed). Click Ok ->Ok.

Path will be set for executing Python programs.

Writing a Python Program:

How to execute python

There are different ways of working in Python:

1) Interactive Mode:

You can enter python in the command prompt and start working with Python.

Line by line execution

Small piece of code executed

2) Script Mode:

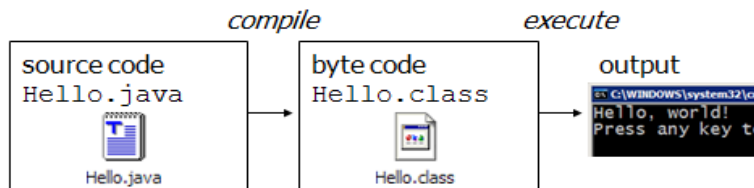
Using Script Mode, you can write your Python code in a separate file using any editor of your Operating System.

Save it by **.py** extension.

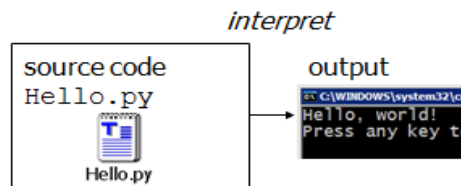
Large program written by using script mode

Compiling and interpreting

- Many languages require you to *compile* (translate) your program into a form that the machine understands.



- Python is instead directly *interpreted* into machine instructions.



1.2 Variables and Types-mutable and Immutable variable and Keywords.

- **Variables:** Variable is a name of the memory location where data is stored. Once a variable is stored that means a space is allocated in memory.

Assigning values to Variable:

- We need not to declare explicitly variable in Python. When we assign any value to the variable that variable is declared automatically.
- The assignment is done using the equal (=) operator.
- **Types:**

Mutable: are those whose values can be changed.

Immutable: are those whose values cannot be changed.

which types support updates and which do not.

Update model category	Python types that fit category
mutable	lists, dictionaries, arrays
immutable	numbers, strings, tuples

- **Example: list is mutable**

We declare list L as follows

```
>>> L = [0,2,3]
```

And then replace zeroth element of list as

```
>>> L[0] = 1
```

And we print the list

```
>>> L
```

```
[1,2,3]
```

Here list L is replace 0 with 1

- **Example: string is immutable**

We declare string s as follows

```
>>> s = 'string'
```

And we just try replace 4th character of string

```
>>> s[3] = 'o'
```

But it will show the `TypeError`

Traceback (most recent call last):

File "<stdin>", line 1, in ?

`TypeError: object does not support item assignment`

So we cannot change the content of string.

Multiple assignment can be done in Python at a time.

Assigning single value to multiple variables:

```
x=y=z=50
```

```
print x
```

```
print y
```

```
print z
```

Assigning multiple values to multiple variables:

```
a,b,c=5,10,15
```

```
print a
```

```
print b
```

```
print c
```

1.3 Operators and Operands in Python.

(Arithmetic, relational and logical operators)

Operators are particular symbols which operate on some values and produce an output.

The values are known as **Operands**.

Example:

$4 + 5 = 9$

Here 4 and 5 are Operands and (+), (=) signs are the operators. They produce the output 9.

- **Python supports the following operators:**

Arithmetic Operators.

Relational Operators.

8 Python Programming for Beginners

Logical Operators.

Membership Operators.

Identity Operators

- **Arithmetic operators:**

Operators	Description
//	Perform Floor division(gives integer value after division)
+	To perform addition
-	To perform subtraction
*	To perform multiplication
/	To perform division
%	To return remainder after division(Modulus)
**	Perform exponent(raise to power)

- **Arithmetic Operator Examples:**

```
>>> 10+20
```

```
30
```

```
>>> 20-10
```

```
10
```

```
>>> 10*2
```

```
20
```

```
>>> 10/2
```

```
5
```

```
>>> 10%3
```

```
1
```

```
>>> 2**3
```

```
8
```

```
>>> 10//3
```

```
3
```

- **Relational Operators:**

Operators	Description
<	Less than
>	Greater than

<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to

- **Relational Operators Examples:**

```
>>> 10<20
```

```
True
```

```
>>> 10>20
```

```
False
```

```
>>> 10<=10
```

```
True
```

```
>>> 20>=15
```

```
True
```

```
>>> 5==6
```

```
False
```

```
>>> 5!=6
```

```
True
```

- **Logical Operators:**

Operators	Description
and	Logical AND(When both conditions are true output will be true)
or	Logical OR (If any one condition is true output will be true)
not	Logical NOT(Compliment the condition i.e., reverse)

- **Logical Operators Examples:**

```
a=5>4 and 3>2
```

```
print a
```

```
True
```

```
b=5>4 or 3<2
```

```
print b
```

```
True
```

```
c=not(5>4)
```

10 Python Programming for Beginners

```
print c
```

```
false
```

- **Membership Operators:**

Operators	Description
in	Returns true if a variable is in sequence of another variable, else false.
not in	Returns true if a variable is not in sequence of another variable, else false.

- **Membership Operators Examples:**

```
a=10
```

```
b=20
```

```
list=[10,20,30,40,50];
```

```
if (a in list):
```

```
    print "a is in given list"
```

```
else:
```

```
    print "a is not in given list"
```

```
if(b not in list):
```

```
    print "b is not given in list"
```

```
else:
```

```
    print "b is given in list"
```

Output:

```
>>>
```

```
a is in given list
```

```
b is given in list
```

- **Identity operators:**

Operators	Description
is	Returns true if identity of two operands are same, else false
is not	Returns true if identity of two operands are not same, else false.

- **Identity operators Examples**

```
a=20
```

```

b=20
if( a is b):
    print "a,b have same identity"
else:
    print "a, b are different"
b=10
if( a is not b):
    print "a,b have different identity"
else:
    print "a,b have same identity"
>>>
a,b have same identity
a,b have different identity

```

1.4 Operator precedence , Expressions and Statements (Assignment statement):

Operator Precedence and Associativity

When different operators appear in the same expression, the normal rules of arithmetic apply. All Python operators have a precedence and associativity:

Precedence—

when an expression contains two different kinds of operators, which should be applied first?

Associativity—

when an expression contains two operators with the same precedence, which should be applied first?

The multiplicative operators (*, /, //, and %) have equal precedence with each other, and the additive operators (binary + and -) have equal precedence with each other.

The multiplicative operators have precedence over the additive operators.

12 Python Programming for Beginners

To see how precedence works, consider the expression

2 + 3 * 4

Should it be interpreted as

(2 + 3) * 4

(that is, 20), or rather is

2 + (3 * 4)

(that is, 14) the correct interpretation? As in normal arithmetic, multiplication and division in Python have equal importance and are performed before addition and subtraction. We say multiplication and division have precedence over addition and subtraction. In the expression

2 + 3 * 4

the multiplication is performed before addition, since multiplication has precedence over addition. The result is 14.

1.5 Taking input (using `raw_input()` and `input()`) and displaying output - `print` statement:

print:

`print` : Produces text output on the console.

Syntax:

`print "Message"`

`print Expression`

Prints the given text message or expression value on the console, and moves the cursor down to the next line.

`print Item1, Item2, ..., ItemN`

Prints several messages and/or expressions on the same line.

Examples:

`print "Hello, world!"`

`age = 45`


```
print "You have", 65 - age, "years until retirement"
```

Output:

```
Hello, world!
```

```
You have 20 years until retirement
```

- **input : Reads a number from user input.**

You can assign (store) the result of input into a variable.

Example:

```
age = input("How old are you? ")
```

```
print "Your age is", age
```

```
print "You have", 65 - age, "years until retirement"
```

Output:

```
How old are you? 53
```

```
Your age is 53
```

```
You have 12 years until retirement
```

- **raw_input : Reads a string of text from user input.**

Example:

```
name = raw_input("Howdy, pardner. What's yer name? ")
```

```
print name, "... what a silly name!"
```

Output:

```
Howdy, pardner. What's yer name? Paris Hilton
```

```
Paris Hilton ... what a silly name!
```

1.6 Comments in Python.

Python supports two types of comments:

1) Single lined comment:

In case user wants to specify a single line comment, then comment must start with #

Example:

```
# This is single line comment.
```

2) Multi lined Comment:

Multi lined comment can be given inside triple quotes.

Example:

```
""" This  
    Is  
    Multiline comment"""
```

Eg.

```
#single line comment
```

```
print "Hello Python"
```

```
"""This is  
multiline comment"""
```

Exercise:

1. Explain the features of python programming.
2. Why python is an interpreted language.
3. Explain operator precedence and associativity.
4. Define the term variable and explain types of variable.
5. Write a program add two numbers provided by the user.
6. Write a program to find the square of number.
7. Write a Python program which accepts the radius of a circle from the user and compute the area.
8. Write a program that takes three numbers and prints their sum. Every number is given on a separate line.
9. Write a program that greets the user by printing the word "Hello", a comma, the name of the user and an exclamation mark after it.
10. Write a program to find the square root of number.

Chapter 2

CONDITIONAL, LOOPING CONSTRUCT AND FUNCTIONS

Conditional and Looping Construct

- ✓ *if - else statement and nested if – else, while, for, use of range function in for, Nested loops*
- ✓ *break, continue, pass statement*
- ✓ *Use of compound expression in conditional constructs*

Functions

- ✓ *Built-In Function, invoking built in functions*
- ✓ *Module (Importing entire module or selected objects using from statement)*
- ✓ *Functions from math, random, time & date module*
- ✓ *Composition*
- ✓ *User Define Function: Defining, invoking functions, passing parameters (default parameter values, keyword arguments)*
- ✓ *Scope of variables, void functions and functions returning values*

2.1 if - else statement and nested if – else while, for, use of range function in for, Nested loops

❖ Conditional Statements

- if
- if...else
- Nested if..elif..else

❖ Looping Statements

- while loop
- for loop
- Nested loops

❖ Iteration Examples

- Multiplication table
- Star tree
- Permutation
- Prime number
- The Guess number game

Story of Two if's:

Consider the following if statement, coded in a **C-like language**:

```
if (p > q)
{
    p = 1;
    q = 2;
}
```

Now, look at the equivalent statement in the **Python language**:

```
if p > q:
    p = 1
    q = 2
```

➤ **what Python adds**➤ **what Python removes****1) Parentheses are optional**

if (x < y)---> if x < y

2) End-of-line is end of statement

C-like languages Python language

x = 1; x = 1

3) End of indentation is end of block**Why Indentation Syntax?****A Few Special Cases**

a = 1; b = 2; print(a + b)

You can chain together only simple statements, like assignments, prints, and function calls.

If Statement

The if statement in Python is same as C language which is used to test a condition.

If condition is true, statement of if block is executed, otherwise it is skipped.

Syntax

if(condition):

 statements

Example:

```
num = 3
if num > 0:
    print(num, "is a positive number.")
print("This is always printed.")
```

```
num = -1
if num > 0:
    print(num, "is a positive number.")
print("This is also always printed.")
```

If else Statements:**Syntax:**

```
if(condition):
    statements
```

else:

```
statements
```

Example:

```
num = 3

# Try these two variations as well.
# num = -5
# num = 0

if num >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

If elif else Statement:

When we need to check for multiple conditions to be true then we use elif Statement.

This statement is like executing a if statement inside a else statement.

Syntax:

If statement:

```
Body
```

elif statement:

```
Body
```

else:

```
Body
```

Example:

```

num = 3.4

# Try these two variations as well:
# num = 0
# num = -4.5

if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")

```

Nested If elif else Statement:

We can have a if...elif...else statement inside another if...elif...else statement.

This is called nesting in computer programming.

```

num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")

```

Example Dog Years:**Write the Python code to address the dog year problem**

It's a generally accepted belief to assume that one year in the life of a dog corresponds to 14 years in the life of a human being. But apparently there are other more subtle methods to calculate this haunting problem. Another subtler method works like this: A one year old dog roughly corresponds to a 14 year old child. A dog who is two years old corresponds to a 22 year old human. Every further dog year corresponds to five human years.

Dog Years Example Solution:

```

age = input("Age of the dog: ")
print
if age < 0:
    print "This can hardly be true!"
elif age == 1:
    print "about 14 human years"
elif age == 2:
    print "about 22 human years"
elif age > 2:
    human = 22 + (age - 2) * 5
    print "Human years: ", human

```

while loop:

The **expression** following the **while** keyword is the **condition** that determines if the statement block is **executed** or continues to execute.

As long as the condition is **true**, the program **executes** the code block over and over again.

When the condition becomes **false**, the loop is **finished**.

If the condition is **false** initially, the code block within the body of the loop is **not executed** at all.

The while statement has the general form:

while condition:

block

Example:

```

entry = 0
sum = 0

print("Enter number")

while entry >= 0:
    entry = input()

```



```

if entry >= 0:
    sum += entry

print("Sum =", sum)

```

While loop examples

Example:1 Letting the User Choose When to Quit

```

prompt = "\nTell me something, and I will repeat it back to you:"

prompt += "\nEnter 'quit' to end the program. "

message = ""

while message != 'quit':
    message =raw_input(prompt)

    print(message)

```

Example:2 Using a Flag

What about complicated programs in which many different events could cause the program to stop running? Trying to test all these conditions in one while statement becomes complicated and difficult.

For example, in a game, several different events can end the game. When the player runs out of ships, their time runs out, or the cities they were supposed to protect are all destroyed, the game should end. It needs to end if any one of these events happens. For a program that should run only as long as many conditions are true, you can define one variable that determines whether or not the entire program is active. This variable, called a ***flag***, *acts as a signal to the program*.

```

prompt = "\nTell me something, and I will repeat it back to you:"

prompt += "\nEnter 'quit' to end the program. "

active=True

while active:
    message =raw_input(prompt)

    if message= 'quit':
        active=False

```

22 Python Programming for Beginners

```
else:  
    print(message)
```

Avoiding Infinite Loops:

Every while loop needs a way to stop running so it won't continue to run forever.

For example, this counting loop should count from **1 to 5**:

```
x = 1  
while x <= 5:  
    print(x)  
    x += 1
```

But if you accidentally **omit the line `x += 1` (as shown next)**, the loop will run forever:

This loop runs forever!

```
x = 1  
while x <= 5:  
    print(x)
```

for loop:

We have used a while loop to implement a definite loop, as in

```
n = 1  
while n <= 10:  
    print(n)  
    n += 1
```

The print statement in this code executes exactly 10 times every time this code runs.

This code requires three crucial pieces to manage the loop:

```
initialization: n = 1  
check: n <= 10  
update: n += 1
```

Python provides a more convenient way to express a definite loop. The for statement iterates over a range of values. These values can be a numeric range or elements of a data structure like a string, list, or tuple.

The above while loop can be rewritten:

```
for n in range(1, 11):
    print(n)
```

for loop:

Repeats a set of statements over a group of values.

Syntax:

for *variableName* in *groupOfValues*:

statements

variableName gives a name to each value, so you can refer to it in the *statements*.

groupOfValues can be a range of integers, specified with the range function.

Example:

```
for x in range(1, 6):
    print x, "squared is", x * x
```

Output:

1 squared is 1

2 squared is 4

3 squared is 9

4 squared is 16

5 squared is 25

The **range** function specifies a range of integers:

range(*start*, *stop*)

- the integers between **start** (inclusive) and **stop** (exclusive)

It can also accept a third value specifying the change between values.

range(*start*, *stop*, *step*)

24 Python Programming for Beginners

- the integers between **start** (inclusive) and **stop** (exclusive) by **step**

Example:

```
for x in range(5, 0, -1):
```

```
    print x
```

```
print "End"
```

Output:

5

4

3

2

1

End

Nested loop

Print a multiplication table to

Print permutations of 3 letter word

Print a Star Tree

Print Prime number

The guess number game

2.2 break, continue, pass statement

break Statement:

Python provides the **break statement** to implement middle-exiting control logic.

During a program's execution, when the **break** statement is encountered within the body of a loop, the remaining statements within the body is **skipped** and the loop is **exited**.

Example:

```
entry = 0
```

```

sum = 0

print("Enter number:")

while True:

    entry = input()

    if entry < 0:

        break

    sum += entry # Add entry to running sum

print("Sum =", sum)

```

continue Statement:

When a **continue statement** is encountered within a loop, the remaining **statements within the body are skipped**, but the loop condition is checked to see if the loop should **continue** or be **exited**.

```

entry = 0

sum = 0

print("Enter number:")

Count=0

while count<=5:

    Count+=1

    entry = input()

    if entry < 0:

        continue

    sum += entry # Add entry to running sum

print("Sum =", sum)

```

pass statement:

There is no limit on the number of statements that can appear in the body of if statement, but there has to be at least one.

It is useful to have a body with no statements.

In that case, you can use the **pass statement**, which does nothing.

```
x=input("Enter number")

if x < 0:

    pass # need to handle negative values!

else:

    print x
```

2.3 Use of Compound Expression in conditional construct

A combination of two or more expressions using logical operators is called a **compound expression**.

Logical AND

If e1 and e2 are two expressions, e1 and e2 are true only if e1 and e2 are both true; if either one is false or both are false, the compound expression is false.

Logical OR

expressions e1 and e2, the compound expression e1 or e2 is false only if e1 and e2 are both false; if either one is true or both are true, the compound expression is true.

Logical NOT

Simply reverses the value of the expression to which it is applied. If e is a true expression, not e is false; if e is false, not e is true.

Both **and** and **or** are **binary operators**; that is, they require two **operands**.

The **not operator** is a **unary** operator.

Operator **not** has higher precedence than both **and** and **or**. **and** has higher precedence than **or**.

short-circuit evaluation:

The **and** operator first tests the expression to its left. If it finds the expression to be false, it does not bother to check the right expression. This approach is called short-circuit evaluation.

In a similar fashion, in the expression **e1 or e2**, if e1 is true, then it does not matter what value e2 has—an or expression is true unless both sub expressions are false.

Why is short-circuit evaluation important? Two situations show why it is important to consider:

1) $(x \neq 0)$ and $(z/x > 1)$ when $x=0$

2) if $x == 1$ or $x == 2$ or $x == 3$:

```
print("OK")
```

Functions:

2.4 Built-In Function, invoking built in functions

Built functions:

Code is made more reusable by packaging it in functions.

A function is a unit of reusable code.

Python provides a collection of standard code stored in libraries called modules.

Programmers can use parts of this library code within their own code to build sophisticated programs.

Introduction to Using Functions:

We have been using functions in Python since the first chapter. These functions include print, input, int, float, range, and type.

In mathematics, a function computes a result from a given value;

for example, from the function definition $f(x)=2x+3$ we can compute $f(5)=13$ and $f(0)=3$.

A function in Python works like a mathematical function.

sqrt() function :

The square root function accepts one numeric (integer or floating-point) value and produces a floating-point result;

```
from math import sqrt

num = input("Enter number: ")

root = sqrt(num);

print("Square root of", num, "=", root)
```

Invocation of Sqrt () function:

The sqrt function can be called many in other ways, as:

```
from math import sqrt

x = 16

# Pass a literal value and display the result
print(sqrt(16.0))

#Pass the variable and display the result
print(sqrt(x))

#Pass an expression
print(sqrt(2 * x - 5))

# Assign result to variable
y = sqrt(x)
print(y)

# Use result in an expression
y = 2 * sqrt(x + 16) - 4
print(y)

# Use result as argument to a function call
y = sqrt(sqrt(256.0))
print(y)

print(sqrt(int('45')))
```

The sqrt function accepts a single numeric argument. The parameter that a client can pass to sqrt can be a literal number, a numeric variable, an arithmetic expression, or even a function invocation that produces a numeric result.

2.5 Functions from Math, Random, Time & Date Module

Standard Mathematical Functions

Table lists only a few of the available functions.

Math functions Module	
sqrt	Computes the square root of a number: $\text{sqrt}(x) = x^{1/2}$
exp	Computes e raised a power: $\text{exp}(x) = e^x$
log10	Computes the common logarithm of a number: $\text{log}(X) = \log_{10} X$
cos	Computes the cosine of a value specified in radians: $\text{cos}(X) = \cos X$;
pow	Raises one number to a power of another: $\text{pow}(x,y) = XY$
degrees	Converts a value in radians to degrees: $\text{degrees}(X) = (\pi/180)*X$
radians	Converts a value in degrees to radians: $\text{radians}(X) = (180/\pi)*X$
fabs	Computes the absolute value of a number: $\text{fabs}(X) = X $

Time Functions:

The time package contains a number of functions that relate to time.

We will consider two: **clock** and **sleep**.

The **clock** function allows us to measure the time of parts of a program's execution. The **clock** returns a floating-point value representing elapsed time in seconds.

measures how long it takes a user to enter a character from the keyboard.

```
from time import clock

Print "Enter your name: ",

start_time = clock()

name = input()

elapsed = clock() - start_time

print(name, "it took you", elapsed, "seconds to respond")
```

measures the time it takes for a Python program to add up all the integers from 1 to 100,000,000.

```
from time import clock

sum = 0

start = clock()
```

30 Python Programming for Beginners

```
for n in range(1, 100000001):  
    sum += n  
  
elapsed = clock() - start  
  
print("sum:", sum, "time:", elapsed)
```

More time efficient prime number program

```
from math import sqrt  
from time import clock  
  
max_value=10000  
count=0  
value=2  
start_time=clock()  
  
while value <=max_value :  
    is_prime=True  
    trial_factor=2  
    root=sqrt(value)  
    while trial_factor<=root:  
        if(value % trial_factor==0):  
            is_prime=False  
            break  
        trial_factor+=1  
    if is_prime:  
        count+=1  
    value+=1  
  
elapsed=clock()-start_time  
  
print "count= ", count, "Elapsed time= ",elapsed
```

sleep function:

The sleep function suspends the program's execution for a specified number of seconds.

Program counts down from 10 with one second intervals between numbers.\

Example:

```
from time import sleep

for count in range(10, -1, -1):

    print(count)

    sleep(1)
```

The sleep function is useful for controlling the speed of graphical animations.

Random Numbers:

Random numbers are useful particularly in games and simulations.

For example, many board games use a die (one of a pair of dice) to determine how many places a player is to advance.

A die or pair of dice are used in other games of chance.

The Python random module contains a number of standard functions that programmers can use for working with pseudorandom numbers. A few of these functions are shown in the Table below.

randomfunctions Module	
random	Returns a pseudorandom floating-point number X in the range $0 \leq X < 1$
randrange	Returns a pseudorandom integer value within a specified range
seed	sets the random number seed

The **seed** function establishes the initial value from which the sequence of pseudorandom numbers is generated. Each call to **random** or **randrange** returns the next value in the sequence of pseudorandom values.

```
from random import randrange, seed

seed(23)

for i in range(0, 100):

    print randrange(1, 1000)
```

```
print()
```

Random Numbers Program:

```
from random import randrange
```

```
for i in range(0, 3):
```

```
    value = randrange(1, 6)
```

```
    print("+-----+")
```

```
if value == 1:
```

```
    print("| |")
```

```
    print("| * |")
```

```
    print("| |")
```

```
elif value == 2:
```

```
    print("| * |")
```

```
    print("| |")
```

```
    print("| * |")
```

```
elif value == 3:
```

```
    print("| * |")
```

```
    print("| * |")
```

```
    print("| * |")
```

```
elif value == 4:
```

```
    print("| * * |")
```

```
    print("| |")
```

```
    print("| * * |")
```

```
elif value == 5:
```

```
    print("| * * |")
```

```
    print("| * |")
```

```
    print("| * * |")
```

```

elif value == 6:
    print("| * * * |")
    print("| |")
    print("| * * * |")
else:
    print(" *** Error: illegal die value ***")
    print("+-----+")

```

2.6 Module (Importing entire module or selected objects using from statement)

Python provides three ways to import functions from a module:

Import one or more specific functions:

```
from math import sqrt, log
```

Import everything the module has to offer:

```
from math import *
```

Import the module itself instead of just its components:

```
import math
```

2.7 Composition:

We can call one function from within another. This ability is called **composition**.

```
x = math.sqrt(sqrt(100))
```

```
x = math.exp(math.log(10.0))
```

2.8 User Defined Function: Defining, invoking functions, passing parameters (*default parameter values, keyword arguments*)

Function Basics:

There are two aspects to every Python function:

Function definition:

The definition of a function contains the code that determines the function's behavior.

Function invocation:

A function is used within a program via a function invocation.

An ordinary function definition consists of three parts:

- **Name—**

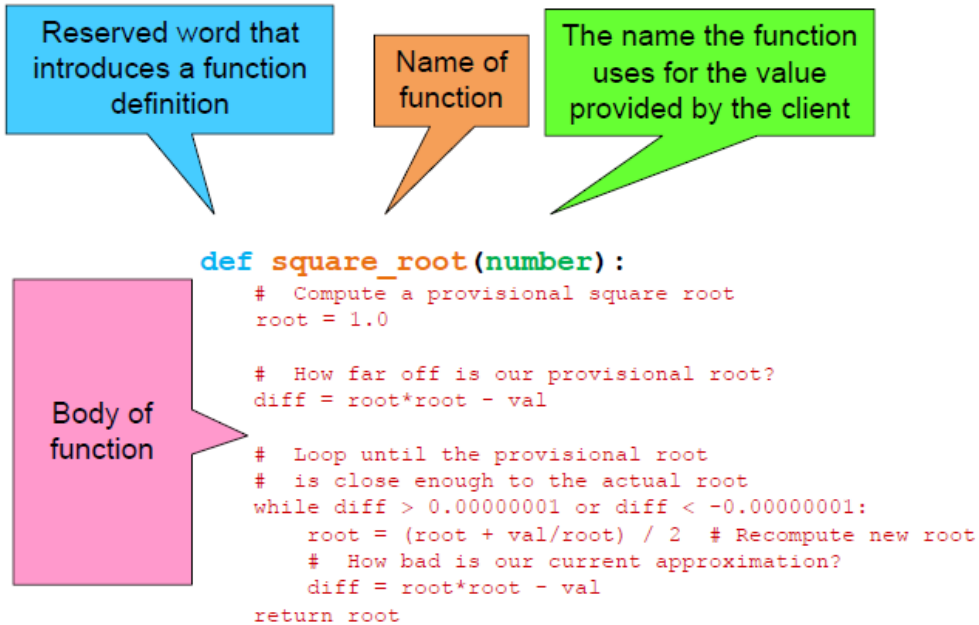
Most Python functions have a name. The name is an identifier.

- **Parameters—**

Every function definition specifies the parameters that it accepts from callers. The parameters appear in a parenthesized comma-separated list. The list of parameters is empty if the function requires no information from code that calls the function.

- **Body—**

Every function definition has a block of indented statements that constitute the function's body. The body contains the code to execute when clients invoke the function. The code within the body is responsible for producing the result.



Examples:

Counts to ten

```
for i in range(1, 11):
    print(i)
```

Count to ten and print each number on its own line

```
def count_to_10():
    for i in range(1, 11):
        print(i)

print("Going to count to ten . . .")
count_to_10()

print("Going to count to ten again. . .")
count_to_10()
```

Count to n and print each number on its own line

```
def count_to_n(n):
```

36 Python Programming for Beginners

```
        for i in range(1, n + 1):  
            print(i)  
  
    print("Going to count to ten . . .")  
    count_to_n(10);  
  
    print("Going to count to five . . .")  
    count_to_n(5);
```

Definition of the prompt function

```
def prompt():  
    value = input("Please enter an integer value: ")  
    return value  
  
print("This program adds together two integers.")  
  
value1 = prompt() # Call the function  
value2 = prompt() # Call the function again  
  
sum = value1 + value2  
  
print(value1, "+", value2, "=", sum)
```

Default Parameters:

Default parameter is a parameter that assumes a default value if a value is not provided in the function call for that parameter.

The following example gives an idea on default parameter:

```
def countdown(n=10):  
    for count in range(n, -1, -1): # Count down from n to zero  
        print(count)
```

The following call

```
countdown()
```

but the invocation

```
countdown(5)
```


We may mix non-default and default parameters in the parameter lists of a function declaration, but all default parameters within the parameter list must appear after all the non-default parameters.

```
def sum_range(n, m=100): # OK, default follows non-default
```

```
    sum = 0
```

```
    for val in range(n, m + 1):
```

```
        sum += val
```

and

```
def sum_range(n=0, m=100): # OK, both default
```

```
    sum = 0
```

```
    for val in range(n, m + 1):
```

```
        sum += val
```

but the following definition is illegal, since a default parameter precedes a non-default parameter in the function's parameter list:

```
def sum_range(n=0, m): # Illegal, non-default follows default
```

```
    sum = 0
```

```
    for val in range(n, m + 1):
```

```
        sum += val
```

Keyword Arguments:

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name.

This allows you to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters.

You can also make keyword calls to the ***printme()** function* in the following ways:

Function definition is here

```
def printme( str ):
```

```
    "This prints a passed string into this function"
```

```
    print str;
```

38 Python Programming for Beginners

```
return;
```

Now you can call printme function

```
printme( str = "My string");
```

When the above code is executed, it produces the following result:

```
My string
```

The following example gives a clearer picture. Note that the order of parameters does not matter.

```
#!/usr/bin/python
```

```
# Function definition is here
```

```
def printinfo( name, age ):
```

```
    "This prints a passed info into this function"
```

```
    print "Name: ", name;
```

```
    print "Age ", age;
```

```
    return;
```

```
# Now you can call printinfo function
```

```
    printinfo( age=50, name="miki" );
```

When the above code is executed, it produces the following result:

```
Name: miki
```

```
Age 50
```

2.9 Scope of Variables, Void Functions and Functions Returning Values Scope of Variables:

Local variable:

Variables defined within functions are local variables.

Local variables have some very desirable properties:

The memory required to store a local variable is used only when the variable is in scope, that is, the variable exists only during the function's execution.

When the program's execution leaves the scope of a local variable, the memory for that variable is freed up. This memory then is reused for the local variables of other functions as needed.

Global variable:

A local variable is transitory, so it disappears in between function invocations.

Sometimes it is desirable to have a variable that exists independent of any function executions.

In contrast to a local variable, a global variable lives outside of all functions and is not local to any particular function. This means any function is capable of accessing and/or modifying a global variable.

The variable is declared to be a global variable using the global reserved word.

```
total = 0; # This is global variable.
```

```
# Function definition is here
```

```
def sum( arg1, arg2 ):
```

```
    # Add both the parameters and return them."
```

```
    total = arg1 + arg2; # Here total is local variable.
```

```
    print "Inside the function local total : ", total
```

```
    return total;
```

```
# Now you can call sum function
```

```
sum( 10, 20 );
```

```
print "Outside the function global total : ", total
```

Void functions and Functions returning values:

Some of the functions we are using, such as the math functions, yield results. Other functions, like print, perform an action but don't return a value. They are called void functions.

Void functions might display something on the screen or have some other effect, but they don't have a return value.

If you try to assign the result to a variable, you get a special value called None.

```
>>> result = print ('Bing')
```

Bing

```
>>> print result
```

None

The value None is not the same as the string 'None'. It is a special value that has its own

type:

```
>>> print type(None)
```

```
<type 'NoneType'>
```

Functions returning values:

The statement `return [expression]` exits a function, optionally passing back an expression to the caller.

A return statement with no arguments is the same as `return None`.

You can return a value from a function as follows:

```
# Function definition is here
def sum( arg1, arg2 ):
    # Add both the parameters and return them."
    total = arg1 + arg2
    print "Inside the function : ", total
    return total;

# Now you can call sum function
total = sum( 10, 20 );
print "Outside the function : ", total
```

Lambda expressions:

*You can use the **lambda keyword** to create small anonymous functions.*

These functions are called anonymous because they are not declared in the standard manner by using the *def keyword*.

Lambda forms can take any number of arguments but return just one value in the form of an expression.

They cannot contain commands or multiple expressions.

Although it appears that lambda's are a one-line version of a function, they are not equivalent to inline statements in C or C++.

The syntax of *lambda functions contains only a single statement, which is as follows:*

lambda [arg1 [,arg2,...,argn]]:expression

Following is the example to show how *lambda form of function works:*

```
#!/usr/bin/python
# Function definition is here

sum = lambda arg1, arg2: arg1 + arg2;

# Now you can call sum as a function

print "Value of total : ", sum( 10, 20 )

print "Value of total : ", sum( 20, 20 )
```

When the above code is executed, it produces the following result:

Value of total : 30

Value of total : 40

Recursive function:

The factorial function is widely used in combinatorial analysis (counting theory in mathematics), probability theory, and statistics. The factorial of n usually is expressed as n!.

Factorial is defined for non-negative integers as

$$n! = n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdots 3 \cdot 2 \cdot 1$$

and 0! is defined to be 1. Thus $6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$. Mathematicians precisely define factorial in this way:

$$n! = \begin{cases} 1, & \text{if } n = 0 \\ n \cdot (n-1)!, & \text{otherwise.} \end{cases}$$

42 Python Programming for Beginners

This definition is recursive since the! function is being defined, but ! is used also in the definition

```
def factorial(n):  
    """  
    Computes n!  
    Returns the factorial of n.  
    """  
  
    if n == 0:  
        return 1  
  
    else:  
        return n * factorial(n - 1)  
  
def main():  
    """ Try out the factorial function """  
    print(" 0! = ", factorial(0))  
    print(" 1! = ", factorial(1))  
    print(" 6! = ", factorial(6))  
    print("10! = ", factorial(10))  
  
main()
```

Observe that the factorial function uses no loop to compute its result.

The factorial function simply calls itself.

Non Recursive function:

```
def factorial(n):  
    """  
    Computes n!  
    Returns the factorial of n.  
    """
```

```

product = 1

while n:

    product *= n

    n -= 1

    return product

def main():

    """ Try out the factorial function """

    print(" 0! = ", factorial(0))

    print(" 1! = ", factorial(1))

    print(" 6! = ", factorial(6))

    print("10! = ", factorial(10))

main()

```

Which factorial function is better, the recursive or non-recursive version? Generally, if both the recursive and non-recursive functions implement the same basic algorithm, the non-recursive function will be more efficient.

A function call is a relatively expensive operation compared to a variable assignment or comparison.

The body of the non-recursive factorial function invokes no functions, but the recursive version calls a function—it calls itself—during all but the last recursive invocation.

The iterative version of factorial is therefore more efficient than the recursive version.

Exercise:

- Q.1 Explain conditional and looping statement in detail?
- Q.2 Explain the three keywords - break, continue and pass in detail.
- Q.3 Explain the functions in Math, Time and Random modules.
- Q.4 Define the term function and explain the term default parameter, keyword argument and lambda expression.
- Q.5 Explain the term scope of variable, void functions and function returning values in detail.

44 Python Programming for Beginners

Q.6 Write a Python program to print multiplication table of n rows and n columns, accept the row and column from the user.

Q.7 Write a Python program to print the prime numbers of up to a given number, accept the number from the user.

Q.8 Write a Python program to guess the number generated by computer.

Q.9 Write a Python program to print the permutation of a three-letter word, accept the word from the user.

Q.10 Write a Python program to print pattern like this:

```

          *
        * * *
      * * * * *
    * * * * * * *
  * * * * * * * *
* * * * * * * * *

          * * * * *
        * * * *
      * * *
    * *
  *

```


Chapter 3

STRINGS

- ✓ *Creating, initializing and accessing the elements*
- ✓ *String operators: +, *, in, not in, range, slice [n:m]*
- ✓ *String built in functions & methods: len, capitalize, find, isalnum, isalpha, isdigit, lower, islower, isupper, upper, lstrip, rstrip, isspace, istitle, partition, replace, join, split, count, decode, encode, swapcase*
- ✓ *Strings constants defined in string module Regular Expression and Pattern Matching*

3.1 Creating, Initializing and Accessing the Elements:

We can create string simply by enclosing characters in single or double quotes.

Creating strings is as simple as assigning a value to a variable.

For example:

```
v1 = 'HelloWorld!'
```

```
v2 = "Python"
```

Accessing Values in Strings:

Python does not support a character type; these are treated as strings of length one, thus also considered a substring.

For example:

```
v1 = 'HelloWorld!'
```

```
v2 = "Python"
```

```
print "v1[0]: ", v1[0]
```

```
print "v2[1:3]: ", v2[1:3]
```

Output:

```
v1[0]: H
```

```
v2[1:3]:yt
```

A string is **a sequence of characters. You can access the characters one at a time with the** bracket operator:

```
>>> fruit = Orange
```

```
>>> letter = fruit[1]
```

The expression in brackets is called an **index. The index indicates which character in the** sequence you want, But you might not get what you expect:

```
>>> print letter
```

```
r
```

For most people, the first letter of 'Orange' is **O, not r.**

But for computer scientists, the index is an offset from the beginning of the string, and the offset of the first letter is zero.

```
>>> letter = fruit[0]
```

```
>>> print letter
```

```
O
```

So O is the 0th letter (“zero-eth”) of ‘Orange’, r is the 1th letter (“one-eth”), and a is the 2th (“two-eth”) letter.

You can use any expression, including variables and operators, as an index, but the value of the index has to be an integer. Otherwise you get:

```
>>> letter = fruit[1.5]
```

TypeError: string indices must be integers, not float

Traversal of string:

One way to write a traversal is with a while loop:

```
index = 0
```

```
while index < len(fruit):
```

```
    letter = fruit[index]
```

```
    print letter
```

```
    index = index + 1
```

This loop traverses the string and displays each letter on a line by itself.

The loop condition **is `index < len(fruit)`**, so when index is equal to the length of the string, the condition is false, and the body of the loop is not executed.

Another way to write a traversal is with a for loop:

```
for char in fruit:
```

```
    print char
```

Q. Write a function that takes a string as an argument and displays the letters backward, one per line.

```
fruit = 'banana'
```

```
index = len(fruit) - 1
```

```

while index >= 0:

    letter = fruit[index]

    print letter

    index = index - 1

```

3.2 String Operators:

Assume string variable **a** holds 'Hello' and variable **b** holds 'python', then:

Operator	Description	Example
+	Concentration - Adds values on either side of the operator	a + b will give HelloPython
*	Repetition - Creates new strings, concatenating multiple copies of the same string	a*2 will give - HelloHello
[]	Slice - Gives the character from the given index	a[1] will give e
[:]	Range Slice - Gives the characters from the given range	a[1:4] will give ell
in	Membership - Returns true if a character exists in the given string	H in a will give 1
not in	Membership - Returns true if a character does not exist in the given string	M not in a will give 1

concatenation operator(+):

The following example shows how to use concatenation (string addition) and a for loop:

In the book MakeWay for Ducklings, the names of the ducklings are Jack, Kack, Lack, Mack, Nack, Ouack, Pack, and Quack. This loop outputs these names in order:

```

prefixes = 'JKLMNOPQ'

suffix = 'ack'

for letter in prefixes:

    print letter + suffix

```

The output is:

Jack

Kack

Lack
Mack
Nack
Oack
Pack
Qack

Of course, that's not quite right because "Ouack" and "Quack" are misspelled.

Exercise: Modify the program to fix this error.

Slice [] operator and range slice [:] operator:

A segment of a string is called a slice.

Selecting a slice is similar to selecting a character:

```
>>> s = 'Monty Python'
>>> print s[0:5]
Monty
>>> print s[6:12]
Python
```

The operator `[n:m]` returns the part of the string from the "n-eth" character to the "m-eth" character, including the first but excluding the last.

If you omit the first index (before the colon), the slice starts at the beginning of the string.

If you omit the second index, the slice goes to the end of the string:

```
>>> fruit = 'banana'
>>> fruit[:3]
'ban'
>>> fruit[3:]
'ana'
```

50 Python Programming for Beginners

If the first index is greater than or equal to the second, the result is an empty string, represented by two quotation marks:

```
>>> fruit = 'banana'
>>> fruit[3:3]
''
```

Exercise: Given that fruit is a string, what does fruit[:] mean?

Strings are immutable:

It is tempting to use the [] operator on the left side of an assignment, with the intention of changing a character in a string.

For example:

```
>>> greeting = 'Hello, world!'
>>> greeting[0] = 'J'
```

TypeError: 'str' object does not support item assignment

An item is one of the values in a sequence.

The reason for the error is that strings are immutable, which means you can't change an existing string.

The best you can do is create a new string that is a variation on the original:

```
>>> greeting = 'Hello, world!'
>>> new_greeting = 'J' + greeting[1:]
>>> print new_greeting
Jello, world!
```

This example concatenates a new first letter onto a slice of greeting. It has no effect on the original string.

in operator:

The word **in** is an operator that takes two strings and returns True if the first appears as a substring in the second:

```
>>> 'a' in 'banana'
```

```
True
```

```
>>> 'seed' in 'banana'
```

```
False
```

Example:

Write a function that prints all the **letters** from **word1** that also appear in **word2**:

```
def letter_match(word1, word2):
```

```
    for letter in word1:
```

```
        if letter in word2:
```

```
            print letter
```

Here's what you get if you compare apples and oranges:

```
>>> letter_match('apples', 'oranges')
```

```
a
```

```
e
```

```
s
```

3.3 String built in functions & methods:

len (): returns length of strings:

len is a built-in function that returns the number of characters in a string:

```
>>> fruit = 'orange'
```

```
>>> len(fruit)
```

```
6
```

To get the last letter of a string, you might be tempted to try something like this:

```
>>> length = len(fruit)
```

```
>>> last = fruit[length]
```

IndexError: string index out of range

The reason for the IndexError is that there is no letter in 'orange' with the index 6. Since we started counting at zero, the six letters are numbered 0 to 5.

To get the last character, you have to subtract 1 from length:

```
>>> last = fruit[length-1]
```

```
>>> print last
```

```
e
```

Alternatively, you can **use negative indices**, which count backward from the end of the string. The expression **fruit[-1]** yields the last letter, **fruit[-2]** yields the second to last, and so on.

capitalize() : Capitalizes first letter of string.

It returns a copy of the string with only its first character capitalized.

Syntax

```
str.capitalize()
```

Example

```
str = "this is string example....wow!!!";
```

```
print "str.capitalize() : ", str.capitalize()
```

Result

```
str.capitalize() : This is string example....wow!!!
```

find(): Determine if substring occurs in a string.

Syntax

```
string.find(str, beg=0 end=len(string))
```

Parameters

str -- This specifies the substring to be searched.

beg -- This is the starting index, by default its 0.

end -- This is the ending index, by default its equal to the length of the string.

Return Value

Index if found and -1 otherwise.

Example 1:

```
str1 = "this is string example....wow!!!";
str2 = "exam";
print str1.find(str2);
print str1.find(str2, 10);
print str1.find(str2, 40);
```

Result

```
15
15
-1
```

Example 2: We invoke find on word and pass the letter we are looking for as a parameter.

```
>>> Word="orange"
>>> word.find('an')
2
```

isalnum() Method: It checks whether the string consists of alphanumeric characters.

Syntax

```
str.isalnum()
```

Return Value

TRUE if all characters in the string are alphanumeric and there is at least one character, FALSE otherwise.

Example

```
str = "this2009"; # No space in this string
print str.isalnum();
```

54 Python Programming for Beginners

```
str = "this is string example....wow!!!";  
print str.isalnum();
```

Result

True

False

isalpha() method: It checks whether the string consists of alphabetic characters only.

Syntax

```
str.isalpha()
```

Return Value:

This method returns true if all characters in the string are alphabetic and there is at least one character, false otherwise.

Example

```
str = "this"; # No space & digit in this string  
print str.isalpha();  
  
str = "this is string example....wow!!!";  
print str.isalpha();
```

Result:

True

False

isdigit()method: The method isdigit() checks whether the string consists of digits only.

Syntax

```
str.isdigit()
```

Return Value

This method returns true if all characters in the string are digits and there is at least one character, false otherwise.

Example

```
str = "123456"; # Only digit in this string
print str.isdigit();

str = "this is string example....wow!!!";
print str.isdigit();
```

Result:

True

False

islower() method: The method `islower()` checks whether all the case-based characters (letters) of the string are lowercase.

Syntax

```
str.islower()
```

Return Value

This method returns true if all cased characters in the string are lowercase and there is at least one cased character, false otherwise.

Example

```
str = "THIS is string example....wow!!!";
print str.islower();

str = "this is string example....wow!!!";
print str.islower();
```

Result:

False

True

isupper() method: The method `isupper()` checks whether all the case-based characters (letters) of the string are uppercase.

Syntax**str.isupper()****Return Value**

This method returns true if all cased characters in the string are uppercase and there is at least one cased character, false otherwise.

Example

```
str = "THIS IS STRING EXAMPLE....WOW!!!";  
print str.isupper()  
  
str = "THIS is string example....wow!!!";  
print str.isupper()
```

Result:

True

False

isspace() Method :

The method isspace() checks whether the string consists of whitespace.

Syntax

```
str.isspace()
```

Return Value

This method returns true if there are only whitespace characters in the string and there is at least one character, false otherwise.

Example:

```
str = " ";  
print str.isspace()  
  
str = "This is string example....wow!!!";  
print str.isspace();
```

Result:

True

False

lower() method: returns a copy of the string in which all case-based characters have been lowercased.

Syntax:

```
str.lower()
```

Return Value

This method returns a copy of the string in which all case-based characters have been lowercased.

Example

```
str = "THIS IS STRING EXAMPLE....WOW!!!";  
print str.lower();
```

Result:

this is string example....wow!!!

Upper () method : returns a copy of the string in which all case-based characters have been uppercased.

Syntax:

```
str.upper()
```

Return Value:

This method returns a copy of the string in which all case-based characters have been uppercased.

Example

```
str = "this is string example....wow!!!";  
print "str.capitalize() : ", str.capitalize()
```

Result:

THIS IS STRING EXAMPLE....WOW!!!

lstrip() method : returns a copy of the string in which all chars have been stripped from the beginning of the string (default whitespace characters).

Syntax

```
str.lstrip([chars])
```

Return Value: This method returns a copy of the string in which all chars have been stripped from the beginning of the string (default whitespace characters).

Example

```
str = " this is string example....wow!!! ";  
print str.lstrip();  
str = "88888888this is string example....wow!!!8888888";  
print str.lstrip('8');
```

Result:

```
this is string example....wow!!!  
  
this is string example....wow!!!8888888
```

rstrip() method: returns a copy of the string in which all chars have been stripped from the end of the string (default whitespace characters).

Syntax:

```
str.rstrip([chars])
```

Return value: This method returns a copy of the string in which all chars have been stripped from the end of the string (default whitespace characters).

Example:

```
str = " this is string example....wow!!! ";  
print str.rstrip();  
str = "88888888this is string example....wow!!!8888888";  
print str.rstrip('8');
```

Result:

```
this is string example....wow!!!  
  
88888888this is string example....wow!!!
```

istitle() method: checks whether all the case-based characters in the string following non-casebased letters are uppercase and all other case-based characters are lowercase.

Syntax

```
str.istitle()
```

Return Value

This method returns true if the string is a titlecased string and there is at least one character,

Example:

```
str = "This Is String Example...Wow!!!";
print str.istitle();
```

Result:

True

False

replace(old, new [, max]): returns a copy of the string in which the occurrences of old have been replaced with new, optionally restricting the number of replacements to max.

Syntax

```
str.replace(old, new[, max])
```

old -- This is old substring to be replaced.

new -- This is new substring, which would replace old substring.

max -- If this optional argument max is given, only the first count occurrences are replaced.

Return Value

This method returns a copy of the string with all occurrences of substring old replaced by new.

Example

```
str = "this is string example....wow!!! this is really string";
print str.replace("is", "was");
```

60 Python Programming for Beginners

```
print str.replace("is", "was", 3);
```

Result:

thwas was string example....wow!!! thwas was really string

thwas was string example....wow!!! thwas is really string

join() method : returns a string in which the string elements of sequence have been joined by str separator.

Syntax

```
str.join(sequence)
```

Return Value: This method returns a string, which is the concatenation of the strings in the sequence seq. The separator between elements is the string providing this method.

Example

```
str = "-";
```

```
seq = ("a", "b", "c"); # This is sequence of strings.
```

```
print str.join( seq );
```

Result:

a-b-c

split(): returns a list of all the words in the string, using str as the separator.

Syntax

```
str.split(str="", num=string.count(str)).
```

Return values:

This method returns a list of lines.

Example:

```
str = "Line1-abcdef \nLine2-abc \nLine4-abcd";
```

```
print str.split( );
```

```
print str.split(' ', 1 );
```


Result:

```
['Line1-abcdef', 'Line2-abc', 'Line4-abcd']
```

```
['Line1-abcdef', '\nLine2-abc \nLine4-abcd']
```

decode() : decodes the string using the codec registered for encoding. It defaults to the default string encoding.

Syntax

```
str.decode(encoding='UTF-8',errors='strict')
```

encoding -- This is the encodings to be used.

errors -- This may be given to set a different error handling scheme. The default for errors is 'strict', meaning that encoding errors raise a UnicodeError.

Return Value

Decoded string.

Example

```
str = "this is string example....wow!!!";
```

```
str = str.encode('base64','strict');
```

```
print "Encoded String: " + str;
```

```
print "Decoded String: " + str.decode('base64','strict')
```

Result

```
Encoded String: dGhpcyBpcyBzdHJpbmcgZXhhbXBsZS4uLi53b3chISE=
```

```
Decoded String: this is string example....wow!!!
```

encode(): returns an encoded version of the string. Default encoding is the current default string encoding.

Syntax

```
str.encode(encoding='UTF-8',errors='strict')
```

encoding -- This is the encodings to be used.

errors -- This may be given to set a different error handling scheme. The default for errors is 'strict'.

Return Value

Encoded string.

Example

```
str = "this is string example....wow!!!";
print "Encoded String: " + str.encode('base64',strict')
```

Result:

Encoded String: dGhpcyBpcyBzdHJpbmcgZXhhbXBsZS4uLi53b3chISE=

swapcase() : returns a copy of the string in which all the case-based characters have had their case swapped.

Syntax

```
str.swapcase();
```

Return Value: This method returns a copy of the string in which all the case-based characters have had their case swapped.

Example

```
str = "this is string example....wow!!!";
print str.swapcase();

str = "THIS IS STRING EXAMPLE....WOW!!!";
print str.swapcase();
```

Result:

'THIS IS STRING EXAMPLE....WOW!!!'

list.count(obj) : returns count of how many times obj occurs in list.

Syntax

```
list.count(obj)
```

Return Value: returns count of how many times obj occurs in list.

Example

```
aList = [123, 'xyz', 'zara', 'abc', 123];
```

```
print "Count for 123 : ", aList.count(123);
print "Count for zara : ", aList.count('zara');
```

Result:

Count for 123 : 2

Count for zara : 1

String comparison :

The relational operators work on strings. To see if two strings are equal:

```
if word == 'banana':
    print 'All right, bananas.'
```

Other relational operations are useful for putting words in alphabetical order:

```
if word < 'banana':
    print 'Your word,' + word + ', comes before banana.'
elif word > 'banana':
    print 'Your word,' + word + ', comes after banana.'
else:
    print 'All right, bananas.'
```

Python does not handle uppercase and lowercase letters the same way that people do. All the uppercase letters come before all the lowercase letters.

String Regular Expressions:

You may be familiar with searching for text by pressing **ctrl-F** and **typing in the words** you're looking for.

Regular expressions go one step further: They allow you to specify a **pattern of text** to search for.

Regular expressions are helpful, but not many non-programmers know about them even though most **modern text editors and word processors**, such as **Microsoft Word** or **OpenOffice**, have find and **find-and-replace** features that can search based on regular expressions.

Regular expressions are huge **time-savers**, not just for software users but also for programmers.

Finding Patterns of Text Without Regular Expressions

Say you want to find a phone number in a string.

You know the pattern:

three numbers, a hyphen, three numbers, a hyphen, and four numbers.

example: 415-555-4242.

Regular expressions, called regexes for short, are descriptions for a pattern of text.

For example, a `\d` in a regex stands for a digit character— that is, any single numeral 0 to 9.

The regex `\d\d\d-\d\d\d-\d\d\d\d` is used by Python to match the expression of three numbers, hyphen, three more numbers, another hyphen, and four numbers.

Any other string would not match the `\d\d\d-\d\d\d-\d\d\d \d\d` regex.

But regular expressions can be much more sophisticated.

Example:

adding a 3 in curly brackets (`{3}`) after a pattern is like saying, “Match this pattern three times.”

So the slightly shorter regex `\d{3}-\d{3}-\d{4}` also matches the correct phone number format.

Symbol and its Meaning:

Symbol	matching
*	0 or more
+	1 or more
?	0 or one
{3}	Exact number
{3,4}	Range of numbers (minimum , maximum)

Quantifiers:

Symbol	matching
*	0 or more
+	1 or more
?	0 or one
{3}	Exact number
{3,4}	Range of numbers (minimum , maximum)

validate the Email from file as well from string by using Regular Expression:

```
# importing the module re
import re
emails=''
CoreyMSchafer@gmail.com
corey.schafer@university.edu
corey-321-schafer@my-work.net

'''
# extract email from string emails
pattern=re.compile(r'([A-Z a-z 0-9 . -]+@[a-z -]+\.(com|edu|net))')
matches =pattern.findall(emails)
for match in matches:
    print(match)

# extract email from file data.txt
pattern=re.compile(r'([A-Z a-z 0-9 . -]+@[a-z -]+\.(com|edu|net))')
with open("data.txt","r") as f:
    contents=f.read()
matches =pattern.findall(contents)
for match in matches:
    print(match)
```

Mobile number validation

```
#importing module re
import re
text_to_search=''
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
1234567890
MetaCharacters (Need to be escaped):
. ^ $ * + ? { } [ ] \ | ( )
coreyms.com
321-555-4321
123.555.1234
236*234*1879
091-9732520247
+91-8615714913
9732520247
'''
#extracting mobile number from string text_to_search
pattern=re.compile(r'[0+]?9?1?[-]?\\d{10}')
matches=pattern.findall(text_to_search)
    for match in matches:
        print (match)

#extracting the mobile from data.txt file
pattern=re.compile(r'[0+]?9?1?[-]?\\d{10}')
with open("data.txt",'r')as f:
    contents=f.read()
    matches=pattern.findall (contents)
    for match in matches:
        print (match)|
```

Url validation

```
import re
urls=''
https://www.google.com
http://sinhgad.edu
https://youtube.com
'''

pattern=re.compile(r'(https?:/(www\.)?[a-z]+\.(com|edu))')
matches =pattern.search(urls)
print (matches)
```

Exercise:

- Q.1 Write a code to create the string and access the elements of string.
- Q.2 Explain the String operators: +, *, in, not in, range, slice [n:m] in details.
- Q.3 Explain the built in functions & methods in string.
- Q.4 Explain the Regular Expression and Pattern Matching in detail.
- Q.5 Write a code to validate mobile number by using regular expressions.
- Q.6 Write a code to validate URL by using regular expressions.
- Q.7 Write a code to validate Email by using regular expressions.

Chapter 4

LISTS, TUPLES, SETS AND DICTIONARIES

Lists:

- ✓ *Concept of Mutable lists, creating, initializing and accessing the elements of list*
- ✓ *List operations (Concatenation, Repetition, Membership, list slices), List comprehensions*
- ✓ *List functions & methods: len, append, extend, sort, remove, reverse, pop*

Tuples:

- ✓ *Immutable concept, creating, initializing and accessing the elements in a tuple;*
- ✓ *Tuple functions: cmp(), len(), max(), min(), tuple()*

Sets:

- ✓ *Concept of Sets, creating, initializing and accessing the elements of sets.*
- ✓ *Sets operation (Membership, union, intersection, difference, and symmetric difference)*

Dictionaries:

- ☑ *Concept of key-value pair, creating, initializing and accessing the elements in a dictionary*
- ☑ *Traversing, appending, updating and deleting elements*
- ☑ *Dictionary functions & Methods: cmp, len, clear(), get(), has_key(), items(), keys(), update(), values().*

4.1 Concept of Mutable Lists, Creating, Initializing and Accessing the Elements of List:

A list is a sequence of values.

In a string, the values are characters; in a list, they can be any type.

The values in a list are called **elements or sometimes items**.

The simplest way is to enclose the elements in square brackets.

```
[100, 200, 300, 400]
```

```
['mango', 'banana', 'apple']
```

The elements of a list **don't have to be the same type**.

The following list contains a string, a float, an integer, and another list:

```
['sham', 5.0, 5, [100, 200]]
```

A list within another list is **nested**.

A list that contains no elements is called an **empty list**;

you can create one with empty brackets, `[]`.

Initializing of Lists:

As you might expect, you can assign list values to variables:

```
>>> fruits = ['apple', 'banana', 'mango']
```

```
>>> numbers = [15, 125]
```

```
>>> empty = []
```

```
>>> print fruits, numbers, empty
```

```
['apple', 'banana', 'mango'] [15, 125] [ ]
```

Lists are mutable:

The syntax for accessing the elements of a list is the same as for accessing the characters of a string—the **bracket operator**.

The expression inside the brackets specifies the index.

Remember that the indices **start at 0**:

```
>>> print fruits[0]
apple
```

When the bracket operator appears on the left side of an assignment, it identifies the element of the list that will be assigned.

```
>>> numbers = [17, 123]
>>> numbers[1] = 5
>>> print numbers
[17, 5]
```

The one-eth element of numbers, which used to be 123, is now 5.

You can think of a list as a relationship between indices and elements. This relationship is called a **mapping**;

each index “maps to” one of the elements.

Traversing a list:

The most common way to traverse the elements of a list is with a for loop.

The syntax is the same as for strings:

Example:

```
for fruit in fruits:
    print fruit
```

4.2 List Operations (Concatenation, Repetition, Membership, List Slices), List Comprehensions

Concatenation:

The + operator concatenates lists:

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print c
[1, 2, 3, 4, 5, 6]
```

Repetitions:

Similarly, the * operator repeats a list a given number of times:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

The first example repeats [0] four times. The second example repeats the list [1, 2, 3] three times.

List slices:

The slice operator also works on lists:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3]
['b', 'c']
>>> t[:4]
['a', 'b', 'c', 'd']
>>> t[3:]
['d', 'e', 'f']
```

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end.

So if you omit both, the slice is a copy of the whole list.

```
>>> t[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

A slice operator on the left side of an assignment can update multiple elements:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3] = ['x', 'y']
>>> print t
['a', 'x', 'y', 'd', 'e', 'f']
```

List Membership:

We can use Python in operator to determine if an object is an element in a list. If `lst` is a list, the expression `x in lst` evaluates to `True` if `x` is an element in `lst`; otherwise, the expression is `False`.

Similarly, the expression `x not in lst` evaluates to `True` if `x` is not an element in `lst`; otherwise, the expression is `False`.

```
lst = list(range(0, 21, 2))
for i in range(-2, 23):
    if i in lst:
        print(i, 'is a member of', lst)
    elif i not in lst:
        print(i, 'is NOT a member of', lst)
```

List Comprehensions:

Mathematicians often represent sets in two different ways:

set roster notation: which enumerates the elements in the set

set builder notation: which describes the contents of the set using a rule.

We can express **P**, the set of **perfect squares less than 50**, using both of these methods:

74 Python Programming for Beginners

We have seen express lists in Python in a manner similar to set roster notation in mathematics:

```
P = [1, 4, 9, 16, 25, 36, 49]
```

Python also supports a technique similar to set builder notation, **called list comprehension**.

In Python, we can express list P above as

```
P = [x**2 for x in [0, 1, 2, 3, 4, 5, 6, 7]]
```

or, using a range expression as

```
P = [x**2 for x in range(8)]
```

Observe the use of the keywords **for** and **in** within the square brackets.

The **for** keyword takes the place of the mathematical $\mid$ symbol (usually pronounced “such that” or “where”), and **in** membership relation.

Example:

The following interactive sequence creates a list containing the first ten multiples of one-half:

```
>>> y = [x/2.0 for x in range(10)]  
  
>>> y  
  
[0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5]
```

4.3 List Functions & Methods: append, extend, sort, remove, reverse, pop:

append():

append adds a new element to the end of a list:

```
>>> t = ['a', 'b', 'c']  
  
>>> t.append('d')  
  
>>> print t  
  
['a', 'b', 'c', 'd']
```

extend ():

takes a list as an argument and appends all of the elements:

```
>>> t1 = ['a', 'b', 'c']  
>>> t2 = ['d', 'e']  
>>> t1.extend(t2)  
>>> print t1  
['a', 'b', 'c', 'd', 'e']
```

Sort ():

arranges the elements of the list from low to high:

```
>>> t = ['d', 'c', 'e', 'b', 'a']  
>>> t.sort()  
>>> print t  
['a', 'b', 'c', 'd', 'e']
```

pop() :**pop delete the elements list**

If you know the index of the element you want to delete, you can use pop:

```
>>> t = ['a', 'b', 'c']  
>>> x = t.pop(1)  
>>> print t  
['a', 'c']  
>>> print x  
b
```

pop modifies the list and returns the element that was removed.

If you don't provide an index, it deletes and returns the last element.

del operator:

If you don't need the **removed value**, you can use the **del operator**:

```
>>> t = ['a', 'b', 'c']
>>> del t[1]
>>> print t
['a', 'c']
```

remove() :

If you know the element you want to remove (but not the index), you can use `remove`:

```
>>> t = ['a', 'b', 'c']
>>> t.remove('b')
>>> print t
['a', 'c']
```

The return value from `remove` is **None**.

To remove more than one element, you can use `del` with a slice index:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> del t[1:5]
>>> print t
['a', 'f']
```

insert() :

inserts object *obj* into list at offset *index*.

Syntax

```
list.insert(index, obj)
```

Return Value

This method does not return any value but it inserts the given element at the given index.

Example

```
aList = [123, 'xyz', 'zara', 'abc']
aList.insert( 3, 2009)
print "Final List : ", aList
```

Output:

```
Final List : [123, 'xyz', 'zara', 2009, 'abc']
```

reverse() :

list.reverse is a method of the list class that mutates an existing list;

The `[::-1]` slice notation returns a new list containing a copy of the original list with its elements in reverse order.

```
lst = [1, 2, 3, 4, 5, 6, 7]
print("---- Original list ----")
print("lst =", lst)
print("---- Slice ----")
obj2 = lst[::-1]
print("lst =", lst)
print("obj2 =", obj2)
print("---- list.reverse method ----")
obj3 = lst.reverse()
print("lst =", lst)
print("obj3 =", obj3)
```

Tuples:

4.4 Immutable concept, creating, initializing and accessing the elements in a tuple:

Concept of Tuple:

A tuple is a sequence of values.

The values can be any type, and they are indexed by integers.

The important difference is that tuples are **immutable**.

Syntactically, a tuple is a comma-separated list of values:

```
>>> t = 'a', 'b', 'c', 'd', 'e'
```

Although it is not necessary, it is common to enclose tuples in parentheses:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

To create a tuple with a single element, you have to include a final comma:

```
>>> t1 = 'a',  
>>> type(t1)  
<type 'tuple'>
```

Most list **operators** also work on tuples.

```
>>> t = ('a', 'b', 'c', 'd', 'e')  
>>> print t[0]  
'a'
```

And the **slice operator selects a range** of elements.

```
>>> print t[1:3]  
( 'b', 'c' )
```

But if you try to modify one of the elements of the tuple, you get an error:

```
>>> t[0] = 'A'  
  
TypeError: object doesn't support item assignment
```

You **can't modify the elements of a tuple**, but you can replace one tuple with another:

```
>>> t = ('A',) + t[1:]

>>> print t

('A', 'b', 'c', 'd', 'e')
```

Tuple initialization:

swapping of value:

For example, to swap **a** and **b**:

```
>>> temp = a

>>> a = b

>>> b = temp
```

This solution is cumbersome; **tuple assignment is more elegant:**

```
>>> a, b = b, a
```

The left side is a tuple of variables; the right side is a tuple of expressions.

The number of variables on the left and the number of values on the right have to be the same:

```
>>> a, b = 1, 2, 3
```

ValueError: too many values to unpack

4.5 Tuple Functions: cmp(), len(), max(), min(), tuple():

Cmp(tuple1, tuple2):

This is used to compare two tuples.

It will return either 1, 0 or -1, depending upon whether the two tuples being compared are similar or not.

The **cmp() function** takes two tuples as arguments, where both of them are compared.

If **T1** is the first tuple and **T2** is the second tuple, then:

if **T1 > T2**, then **cmp(T1, T2)** returns **1**

80 Python Programming for Beginners

if **T1 = T2**, then `cmp(T1, T2)` returns **0**

if **T1 < T2**, then `cmp(T1, T2)` returns **-1**

Example:

The following example shows the usage of `cmp()` method.

```
tuple1, tuple2 = (123,), (456,)
print cmp(tuple1, tuple2);
print cmp(tuple2, tuple1);
```

Output:

-1

1

len(tuple):

The method **len()** returns the number of elements in the tuple.

Syntax

```
len(tuple)
```

Return Value

This method returns the number of elements in the tuple.

Example:

```
tuple1, tuple2 = (123, 'xyz', 'pqr'), (456, 'abc')
print "First tuple length: ", len(tuple1);
print "Second tuple length: ", len(tuple2);
```

Output:

First tuple length : 3

Second tuple length : 2

max(tuple):

The method **max()** returns the elements from the tuple with maximum value.

syntax

```
max(tuple)
```

Return Value

This method returns the elements from the tuple with maximum value.

Example:

```
tuple1, tuple2 = (123, 'xyz', 'pqr', 'abc'), (456, 700, 200)
print "Max value element : ", max(tuple1);
print "Max value element : ", max(tuple2);
```

Output:

```
Max value element : xyz
Max value element : 700
```

min(tuple):

The method **min()** returns the elements from the tuple with minimum value.

Syntax:

```
min(tuple)
```

Return Value

This method returns the elements from the tuple with minimum value.

Example:

```
tuple1, tuple2 = (123, 'xyz', 'pqr', 'abc'), (456, 700, 200)
print "min value element : ", min(tuple1);
print "min value element : ", min(tuple2);
```

Output:

```
min value element : 123
min value element : 200
```

tuple(seq):**Syntax:**

```
tuple( seq )
```

Return Value

This method returns the tuple.

Example:

The following example shows the usage of tuple() method.

```
x=tuple("xyz")
print x
```

Output:

```
('x', 'y', 'z')
```

Sets:**4.6 Concept of Sets, Creating, Initializing and Accessing the Elements of Sets:**

We use curly braces ({ }) in Python code to enclose the elements of a set.

Unlike Python lists, sets are unordered and may contain no duplicate elements.

The following interactive sequence demonstrates these set properties:

```
>>> S = {10, 3, 7, 2, 11}
>>> S
{2, 11, 3, 10, 7}
>>> T = {5, 4, 5, 2, 4, 9}
>>> T
{9, 2, 4, 5}
```

element ordering of the input is different from the ordering in the output.

Also sets do not admit duplicate elements.

We can make a set out of a list using the set conversion function:

```
>>> L = [10, 13, 10, 5, 6, 13, 2, 10, 5]
>>> S = set(L)
>>> L
[10, 13, 10, 5, 6, 13, 2, 10, 5]
>>> S
{10, 2, 13, 5, 6}
```

As you can see, the element ordering is not preserved, and duplicate elements appear only once in the set.

Python set notation exhibits one important difference with mathematics:

the expression {} does not represent the empty set.

4.7 Sets Operation (Membership, Union, Intersection, Difference, and Symmetric Difference):

Python supports the standard mathematical set operations of **intersection**, **union**, **set difference**, and **symmetric difference**.

Operation	Mathematical Notation	Python Syntax	Result Type	Meaning
Union	$A \cup B$	$A \mid B$	set	Elements in A or B or both
Intersection	$A \cap B$	$A \& B$	set	Elements common to both A and B
Set Difference	$A - B$	$A - B$	set	Elements in A but not in B
Symmetric Difference	$A \oplus B$	$A \wedge B$	set	Elements in A or B , but not both
Set Membership	$x \in A$	$x \text{ in } A$	bool	x is a member of A
Set Membership	$x \notin A$	$x \text{ not in } A$	bool	x is not a member of A
Set Equality	$A = B$	$A == B$	bool	Sets A and B contain exactly the same elements
Subset	$A \subseteq B$	$A \leq B$	bool	Every element in set A also is a member of set B
Proper Subset	$A \subset B$	$A < B$	bool	A is a subset B , but B contains at least one element not in A

The following interactive sequence computes the set operations on two sets

```
>>> S = {2, 5, 7, 8, 9, 12}
```

```
>>> T = {1, 5, 6, 7, 11, 12}
```

Set Union operation:

Combination of elements in both the sets.

```
>>> S | T
```

```
{1, 2, 5, 6, 7, 8, 9, 11, 12}
```

Set Intersection operation:

Elements common in both S and T sets.

```
>>> S & T
```

```
{12, 5, 7}
```

Set Membership operation:

It checks whether the element is the member of set or not.

```
>>> 7 in S
```

```
True
```

```
>>> 11 in S
```

```
False
```

Set Difference operation:

Elements in S but not in T

```
>>> S-T
```

```
{2,8,9}
```

Set Symmetric operation:

Elements in S or T but not in both

```
>>> S^T
```

```
{1,2,6,8,11}
```


Dictionary:

4.8 Concept of Key-Value Pair, Creating, Initializing and Accessing the Elements in a Dictionary:

Creating, Initializing and Accessing the Dictionary:

A **dictionary is like a list, but more general. In a list, the indices have to be integers; in a dictionary they can be any type.**

You can think of a dictionary as a **mapping** between a set of indices (which are called **keys**) and a set of values. Each key maps to a value. The association of a key and a value is called a **key-value pair**.

Creation of Dictionary:

Example:

Build a dictionary that maps from **English to Spanish** words, so the **keys and the values are all strings**.

The function **dict** creates a new dictionary with no items.

```
>>> eng2sp = dict()
>>> print eng2sp
{ }
```

The curly-brackets, **{ }**, represent an **empty** dictionary.

To add items to the dictionary, you can use **square brackets**:

```
>>> eng2sp['one'] = 'uno'
```

This line creates an item that maps from the **key 'one'** to the **value 'uno'**.

If we print the dictionary again, we see a **key-value pair** with a **colon** between the key and value:

```
>>> print eng2sp
{'one': 'uno'}
```

The output format is also an input format.

Example, you can create a new dictionary with three items:

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

But if you print `eng2sp`, you might be surprised:

```
>>> print eng2sp  
{'one': 'uno', 'three': 'tres', 'two': 'dos'}
```

The order of the **key-value pairs is not the same**.

In general, the order of items in a dictionary is unpredictable.

Order of item:

```
>>> print eng2sp['two']  
'dos'
```

The key **'two'** always maps to the value **'dos'** so the order of the items doesn't matter.

If the **key** isn't in the dictionary, you get an exception:

```
>>> print eng2sp['four']  
KeyError: 'four'
```

Methods of Dictionary:

The **len** function works on dictionaries; it returns the number of **key-value pairs**:

```
>>> len(eng2sp)  
3
```

The **in operator** works on dictionaries; it tells you whether something appears as a key in the dictionary.

```
>>> 'one' in eng2sp  
True  
  
>>> 'uno' in eng2sp  
False
```

To see whether something appears as a value in a dictionary, you can use the **values()**: which returns the values as a list, and then use the **in** operator:

```
>>> vals = eng2sp.values()
```

```
>>> 'uno' in vals
```

```
True
```

4.9 Traversing, Appending, Updating and Deleting Elements:

Updating and deleting the Dictionary:

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry.

Example:

```
dict = {'Name': 'Sara', 'Age': 7, 'Class': 'First'};
dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']:", dict['Age'];
print "dict['School']:", dict['School'];
```

Output:

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements:

You can either remove individual dictionary elements or clear the entire contents of a dictionary.

```
dict = {'Name': 'Sara', 'Age': 7, 'Class': 'First'};
del dict['Name']; # remove entry with key 'Name'
dict.clear(); # remove all entries in dict
del dict ; # delete entire dictionary
```

This produces the following result Note that an exception is raised because after **del dict, dictionary does not exist anymore**

```
dict['Age']:
```

```
Traceback (most recent call last):
```

```
File "test.py", line 8, in <module>
```

```
print "dict['Age']:", dict['Age'];
```

```
TypeError: 'type' object is unsubscriptable
```

4.10 Dictionary Functions & Methods: **cmp**, **len**, **clear()**, **has_key()**, **items()**, **keys()**, **update()**, **values()**:

cmp(dict1, dict2):

The method **cmp()** compares two dictionaries based on key and values.

Syntax

```
cmp(dict1, dict2)
```

Return Value

This method returns 0 if both dictionaries are equal, -1 if dict1 < dict2, and 1 if dict1 > dict2.

Example:

```
dict1 = {'Name': 'Sara', 'Age': 7};
dict2 = {'Name': 'Rahul', 'Age': 27};
dict3 = {'Name': 'Ankush', 'Age': 27};
dict4 = {'Name': 'Sara', 'Age': 7};
print "Return Value : %d" % cmp (dict1, dict2)
print "Return Value : %d" % cmp (dict2, dict3)
print "Return Value : %d" % cmp (dict1, dict4)
```

Output:

```
Return Value : -1
```

```
Return Value : 1
```

```
Return Value : 0
```

clear() :

The method **clear()** removes all items from the dictionary

Syntax

Following is the syntax for **clear()** method:

```
dict.clear()
```

Return Value

This method does not return any value

Example

The following example shows the usage of clear() method

```
dict = {'Name': 'Zara', 'Age': 7};  
print "Start Len : %d" % len(dict)  
dict.clear()  
print "End Len : %d" % len(dict)
```

Output:

```
Start Len : 2  
End Len : 0
```

has_key(key) :

The method **has_key()** returns true if a given *key is available in the dictionary*, otherwise it returns a false.

Syntax

```
dict.has_key(key)
```

Return Value

This method returns true if a given key is available in the dictionary, otherwise it returns a false.

Example

The following example shows the usage of has_key() method.

```
dict = {'Name': 'Sara', 'Age': 7}
```

90 Python Programming for Beginners

```
print "Value : %s" % dict.has_key('Age')  
print "Value : %s" % dict.has_key('Gender')
```

Output:

```
Value : True  
Value : False
```

items():

The method **items()** returns a list of dict's (key, value) tuple pairs

Syntax

```
dict.items()
```

Return Value

This method returns a list of tuple pairs.

Example

```
dict = {'Name': 'Sara', 'Age': 7}  
print "Value : %s" % dict.items()
```

Output:

```
Value : [('Age', 7), ('Name', 'Sara')]
```

keys() :

The method **keys()** returns a list of all the available keys in the dictionary.

Syntax

```
dict.keys()
```

Return Value

This method returns a list of all the available keys in the dictionary.

Example

```
dict = {'Name': 'Sara', 'Age': 7}  
print "Value : %s" % dict.keys()
```

Output:

```
Value : ['Age', 'Name']
```

update(dict2) :

The method **update()** adds dictionary **dict2**'s key-values pairs in to **dict**. This **function** does not return anything.

Syntax

```
dict.update(dict2)
```

Return Value

This method does not return any value.

Example

```
dict = {'Name': 'Sara', 'Age': 7}
dict2 = {'Gender': 'female' }
dict.update(dict2)
print "Value : %s" % dict
```

Output:

```
Value : {'Age': 7, 'Name': 'Sara', 'Gender': 'female'}
```

values():

The method **values()** returns a list of all the values available in a given dictionary.

Syntax

```
dict.values()
```

Return Value

This method returns a list of all the values available in a given dictionary.

Example

```
dict = {'Name': 'Sara', 'Age': 7}
print "Value : %s" % dict.values()
```

Output:

Value : [7, 'Sara']

Exercise:

- Q.1 What is dictionary? Write a Python code to create dictionary. Perform `cmp()`, `len()`, `get()`, and `has_key()` operations on it.
- Q.2 What is list? Write a Python code to create list. Perform `len`, `append`, `extend`, `sort`, `remove`, `reverse`, `pop` on it.
- Q.3 What is tuple and explain `cmp()`, `len()`, `max()`, `min()`, `tuple()` methods of tuples.
- Q.4 Explain the concept of sets and perform set operations Membership, union, intersection, difference, and symmetric difference.
- Q.5 Write Python code to show list is mutable and tuple and string are immutable.
- Q.6 Explain the List comprehension concept in detail.
- Q.7 Write a Python code to convert lists into tuples.

Chapter 5

MODULE

- ☑ *Concept of Module: Executing modules as scripts, The Module Search Path, “Compiled” Python files, Standard Modules*
- ☑ *The `dir()` Function*
- ☑ *Packages Importing * From a Package, Intra-package References, Packages in Multiple Directories*

5.1 Concept of Module: Executing Modules as Scripts, the Module Search Path, “Compiled” Python Files, Standard Modules: What is Module?

If you quit from the Python interpreter and enter it again, the definitions you have made (functions and variables) are lost.

Therefore, if you want to write a somewhat longer program, you are better off using a text editor to prepare the input for the interpreter and running it with that file as input instead. This is known as creating a script.

You may also want to use a handy function that you’ve written in several programs without copying its definition into each program.

To support this, Python has a way to put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a **module**;

Definitions from a module can be imported into other modules or into the main module.

Within a module, the module’s name (as a string) is available as the value of the global variable `__name__`.

Example:

```
# Fibonacci numbers module
```

```
def fib(n): # write Fibonacci series up to n
```

```
    a, b = 0, 1
```

```
    while b < n:
```

```
        print b,
```

```
        a, b = b, a+b
```

Now enter the Python interpreter and import this module with the following command:

```
>>> import fibo
```

Using the module name, you can access the functions:

```
>>> fibo.fib(1000)
```

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

Executing Module as a script:

When you run a Python module with

```
python fibo.py <arguments>
```

The code in the module will be executed, just as if you imported it, but with the `__name__` set to `"__main__"`.

That means that by adding this code at the end of your module:

```
if __name__ == "__main__":
    fib(10)
```

you can make the file usable as a script as well as an importable module, because the code that parses the command line only runs if the module is executed as the “main” file:

```
$ python fibo.py 50
```

```
1 1 2 3 5 8 13 21 34
```

If the module is imported, the code is not run:

```
>>> import fibo
```

```
>>>
```

The Module Search Path:

When a module named `fibo` is imported, the interpreter first searches for a built-in module with that name. If not found, it then searches for a file named `fibo.py` in a list of directories given by the variable `sys.path`.

`sys.path` is initialized from these locations:

- the directory containing the input script (or the current directory).
- `PYTHONPATH`
- the installation-dependent default.

After initialization, Python programs can modify `sys.path`.

The directory containing the script being run is placed at the beginning of the search path, ahead of the standard library path.

This means that scripts in that directory will be loaded instead of modules of the same name in the library directory.

“Compiled” Python files:

As an important speed-up of the start-up time for short programs that use a lot of standard modules, if a file called `fibonacci.pyc` exists in the directory where `fibonacci.py` is found, this is assumed to contain an already- “byte-compiled” version of the module `fibonacci`.

The modification time of the version of `fibonacci.py` used to create `fibonacci.pyc` is recorded in `fibonacci.pyc`, and the `.pyc` file is ignored if these don’t match.

Normally, you don’t need to do anything to create the `fibonacci.pyc` file. Whenever `fibonacci.py` is successfully compiled, an attempt is made to write the compiled version to `fibonacci.pyc`.

It is not an error if this attempt fails; if for any reason the file is not written completely, the resulting file will be recognized as invalid and thus ignored later.

The contents of the `fibonacci.pyc` file are platform independent, so a Python module directory can be shared by machines of different architectures.

Standard Module:

Python comes with a library of standard modules, described in a separate document, the Python Library Reference.

Some modules are built into the interpreter;

These provide access to operations that are not part of the core of the language but are nevertheless built in, or for to provide access to operating system primitives such as `system` call.

The set of such modules is a configuration option which also depends on the underlying platform.

For example, the **`winreg`** module is only provided on Windows systems.

One particular module deserves some attention:

`sys`, which is built into every Python interpreter.

The variables `sys.ps1` and `sys.ps2` define the strings used as primary and secondary prompts:

```
>>> import sys
```

```
>>> sys.ps1
```

```
'>>> '
```

```
>>> sys.ps2
```

```
'... '
```

5.2 The `dir()` Function:

The built-in function `dir()` is used to find out which names a module defines.

It returns a sorted list of strings:

```
>>> import fibo, sys
```

```
>>> dir(fibo)
```

```
['__name__', 'fib', 'fib2']
```

```
>>> dir(sys)
```

```
['__displayhook__', '__doc__', '__excepthook__', '__name__', '__package__', '__stderr__', '__stdin__', '__stdout__', '_clear_type_cache', '_current_frames', '_getframe', '_mercurial', 'api_version', 'argv', 'builtin_module_names', 'byteorder', 'call_tracing', 'callstats',
```

```
'copyright', 'displayhook', 'dont_write_bytecode', 'exc_clear', 'exc_info', 'exc_traceback', 'exc_type', 'exc_value', 'excepthook', 'exec_prefix', 'executable', 'exit', 'flags', 'float_info', 'float_repr_style',
```

```
'getcheckinterval', 'getdefaultencoding', 'getdlopenflags', 'getfilesystemencoding', 'getobjects', 'getprofile', 'getrecursionlimit', 'getrefcount', 'getsizeof', 'gettotalrefcount', 'gettrace', 'hexversion', 'long_info', 'maxint', 'maxsize', 'maxunicode', 'meta_path', 'modules',
```

```
'path', 'path_hooks', 'path_importer_cache', 'platform', 'prefix', 'ps1', 'py3kwarning', 'setcheckinterval', 'setdlopenflags', 'setprofile', 'setrecursionlimit', 'settrace', 'stderr', 'stdin', 'stdout', 'subversion', 'version', 'version_info', 'warnoptions']
```

Without arguments, `dir()` lists the names you have defined currently:

```
>>> a = [1, 2, 3, 4, 5]
```

```
>>> import fibo
```

```
>>> fib = fibo.fib
```

```
>>> dir()
```

```
['__builtins__', '__name__', '__package__', 'a', 'fib', 'fibo', 'sys']
```

Note that it lists all types of names: variables, modules, functions, etc.

`dir()` does not list the names of built-in functions and variables.

If you want a list of those, they are defined in the standard module `__builtin__`:

```
>>> import __builtin__
```

```
>>> dir(__builtin__)
```

```
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BufferError',
'BytesWarning', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EnvironmentError',
'Exception', 'False', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError',
'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'KeyError',
'KeyboardInterrupt',
```

```
'LookupError', 'MemoryError', 'NameError', 'None', 'NotImplemented',
'NotImplementedError', 'OSError', 'OverflowError',
```

```
'PendingDeprecationWarning', 'ReferenceError', 'RuntimeError', 'RuntimeWarning',
'StandardError', 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError',
'SystemExit', 'TabError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError',
'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning',
'UserWarning', 'ValueError', 'Warning', 'ZeroDivisionError', '_', '__debug__', '__doc__',
'__import__', '__name__', '__package__', 'abs', 'all', 'any', 'apply', 'basestring', 'bin',
'bool', 'buffer', 'bytearray', 'bytes', 'callable', 'chr', 'classmethod', 'cmp', 'coerce', 'compile',
'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'execfile',
'exit', 'file', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help',
'hex', 'id', 'input', 'int', 'intern', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals',
'long', 'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print',
'property', 'quit', 'range', 'raw_input', 'reduce', 'reload', 'repr', 'reversed', 'round', 'set',
'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'unichr', 'unicode',
'vars', 'xrange', 'zip']
```

5.3 Package

Packages are a way of structuring Python's module namespace by using **“dotted module names”**.

Example:

The module name `A.B` designates a submodule named `B` in a package named `A`.

Suppose you want to design a collection of modules (a “package”) for the uniform handling of sound files and sound data.

There are many different sound file formats (usually recognized by their extension).

Example:

.wav, .aiff, .au, so you may need to create and maintain a growing collection of modules for the conversion between the various file formats. There are also many different operations you might want to perform on sound data (such as mixing, adding echo, applying an equalizer function, creating an artificial stereo effect), so in addition, you will be writing a never-ending stream of modules to perform these operations.

```

sound/                                Top-level package
  __init__.py                          Initialize the sound package
  formats/                             Subpackage for file format conversions
    __init__.py
    wavread.py
    wavwrite.py
    aiffread.py
    aiffwrite.py
    auread.py
    auwrite.py
    ...
  effects/                             Subpackage for sound effects
    __init__.py
    echo.py
    surround.py
    reverse.py
    ...
  filters/                             Subpackage for filters
    __init__.py
    equalizer.py
    vocoder.py
    karaoke.py
    ...

```

Importing * From a Package:

Now what happens when the user writes `from sound.effects import *`?

Ideally, one would hope that this somehow goes out to the filesystem, finds which submodules are present in the package, and imports them all.

This could take a long time and importing sub-modules might have unwanted side-effects that should only happen when the sub-module is explicitly imported. The only solution is for the package author to provide an explicit index of the package.

The import statement uses the following convention:

100 Python Programming for Beginners

if a package's `__init__.py` code defines a list named `__all__`, it is taken to be the list of module names that should be imported when `from package import *` is encountered.

For example, the file `sound/effects/__init__.py` could contain the following code:

```
__all__ = ["echo", "surround", "reverse"]
```

This would mean that `from sound.effects import *` would import the three named submodules of the `sound` package.

If `__all__` is not defined, the statement `from sound.effects import *` does not import all submodules from the package `sound.effects` into the current namespace; it only ensures that the package `sound.effects` has been imported and then imports whatever names are defined in the package.

Consider this code:

```
import sound.effects.echo

import sound.effects.surround

from sound.effects import *
```

Example:

The `echo` and `surround` modules are imported in the current namespace because they are defined in the `sound.effects` package when the `from...import` statement is executed.

Intra-package References:

The submodules often need to refer to each other.

Example, the `surround` module might use the `echo` module.

In fact, such references are so common that the `import` statement first looks in the containing package before looking in the standard module search path.

Thus, the `surround` module can simply use `import echo` or `from echo import echofilter`.

If the imported module is not found in the current package, the `import` statement looks for a top-level module with the given name.

When packages are structured into subpackages, you can use absolute imports to refer to submodules of siblings packages.

For example, if the module `sound.filters.vocoder` needs to use the `echo` module in the `sound.effects` package, it can use `from sound.effects import echo`.

Explicit relative imports use leading dots to indicate the current and parent packages involved in the relative import.

From the surround module for example, you might use:

```
from . import echo  
  
from .. import formats  
  
from ..filters import equalizer
```

Packages in Multiple Directories:

Packages support one more special attribute, `__path__`.

This is initialized to be a list containing the name of the directory holding the package's `__init__.py` before the code in that file is executed.

This variable can be modified; doing so affects future searches for modules and subpackages contained in the package. While this feature is not often needed, it can be used to extend the set of modules found in a package.

Exercise:

Q.1 Explain the concept of Module in detail with examples.

Q.2 Write a Python code to execute module as script.

Q.3 Explain the `dir ( )` Function in detail.

Q.4 What is package? Write a Python code to create package of FRUIT and create two modules APPLE and ORANGE in it and it contains apple and orange classes respectively. Create test script.py file to access both the module in it.

Q.5 Write short notes :

- a) Standard module
- b) Intra package references
- c) Module search path

Chapter 6

IO AND FILE HANDLING

- ✓ *Output Formatting*
- ✓ *Filenames and Paths*
- ✓ *Reading and Writing Files(text and binary mode)*
- ✓ *Storing complex data in files*
- ✓ *Exercise 6*

Persistence:

Most of the programs we have seen so far are transient in the sense that they run for a short time and produce some output, but when they end, their data disappears.

If you run the program again, it starts with a clean slate.

Other programs are persistent: they run for a long time (or all the time); they keep at least some of their data in permanent storage (a hard drive, for example); and if they shut down and restart, they pick up where they left off.

Examples of persistent programs are operating systems, which run pretty much whenever a computer is on, and web servers, which run all the time, waiting for requests to come in on the network.

One of the simplest ways for programs to maintain their data is by reading and writing text files.

6.1 Output Formatting:

Printing to the Screen:

The simplest way to produce output is using the ***print*** statement where you can pass zero or more expressions separated by commas.

This function converts the expressions you pass into a string and writes the result to standard output as follows:

```
print "Python is really a great language,", "isn't it?";
```

This produces the following result on your standard screen:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input:

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard.

These functions are:

raw_input

input

The `raw_input` Function:

The `raw_input([prompt])` function reads one line from standard input and returns it as a string.

Example:

```
str = raw_input("Enter your input: ");
print "Received input is : ", str
```

This would produce the following result against the entered input:

```
Enter your input: Hello Python
Received input is : Hello Python
```

input() function : is equivalent to `raw_input`, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

Example:

```
str = input("Enter your input: ");
print "Received input is : ", str
```

This would produce the following result against the entered input:

```
Enter your input: [x*5 for x in range(2,10,2)]
Received input is : [10, 20, 30, 40]
```

The argument of **write** has to be a string, so if we want to put other values in a file, we have to convert them to strings. The easiest way to do that is with **str**:

```
>>> x = 52
>>> fout.write(str(x))
```

An alternative is to use the format operator, `%`. When applied to integers, `%` is the modulus operator. But when the first operand is a string, `%` is the format operator.

The first operand is the format string, which contains one or more format sequences, which specify how the second operand is formatted.

The result is a string.

For example, the format sequence `'%d'` means that the second operand should be formatted as an integer (`d` stands for "decimal"):

```
>>> camels = 42

>>> '%d' % camels

'42'
```

The result is the string '42', which is not to be confused with the integer value 42.

A format sequence can appear anywhere in the string, so you can embed a value in a sentence:

```
>>> camels = 42

>>> 'I have spotted %d camels.' % camels

'I have spotted 42 camels.'
```

If there is more than one format sequence in the string, the second argument has to be a tuple. Each format sequence is matched with an element of the tuple, in order. The following example uses **'%d'** to format an integer, **'%g'** to format a floating-point number and **'%s'** to format a string:

```
>>> 'In %d years I have spotted %g %s.' % (3, 0.1, 'camels')

'In 3 years I have spotted 0.1 camels.'
```

The number of elements in the tuple has to match the number of format sequences in the string. Also, the types of the elements have to match the format sequences:

```
>>> '%d %d %d' % (1, 2)

TypeError: not enough arguments for format string

>>> '%d' % 'dollars'

TypeError: illegal argument type for built-in operation
```

6.2 Filenames and Paths:

You can do most of the file manipulation using a file object.

The open Function:

Before you can read or write a file, you have to open it using Python's builtin

open() function.

*This function creates a **file object**, which would be utilized to call other support methods associated with it.*

Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

file_name:

The file_name argument is a string value that contains the name of the file that you want to access.

access_mode:

The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. The default file access mode is read (r).

buffering:

If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default.

Modes of Opening file:

Here is a list of the different modes of opening a file:

Models	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer is placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer is placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.

a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

file Object Attributes:

Once a file is opened and you have one *file object*, you can get various information related to that file. Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

```
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
print "Closed or not : ", fo.closed
print "Opening mode : ", fo.mode
print "Softspaceflag : ", fo.softspace
```

This produces the following result:

```
Name of the file:  foo.txt
Closed or not :   False
Opening mode :    wb
Softspace flag :  0
```

close() Method:

The close() method of a *file object* flushes any unwritten information and closes the file object, after which no more writing can be done.

Syntax

```
fileObject.close();
```

Example

```
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
fo.close()
```

This produces the following result:

Name of the file: foo.txt

6.3 Reading and Writing Files:

write() Method:

The *write()* method writes any string to an open file.

Syntax

```
fileObject.write(string);
```

Example

```
fo = open("foo.txt", "wb")
fo.write("Python is a great language.\nYeah its great!!\n");
fo.close()
```

The above method would create *foo.txt* file and would write given content in that file and finally it would close that file.

If you would open this file, it would have the following content.

```
Python is a great language.
Yeah its great!!
```

read () method:**File.read([size])**

The method *read()* reads at most *size* bytes from the file.

If the read hits EOF before obtaining *size* bytes, then it reads only available bytes.

Syntax

```
fileObject.read( size );
```

Parameters

size -- This is the number of bytes to be read from the file.

Return Value

This method returns the bytes read in string.

Example

```
fo = open("foo.txt", "r")  
line = fo.read(10)  
print "Read Line: %s" % (line)  
fo.close()
```

readline() method:

File.readline([size])

The method **readline()** reads one entire line from the file.

Syntax

```
fileObject.readline( size );
```

Parameters

size -- This is the number of bytes to be read from the file.

Return Value

This method returns the line read from the file.

Example

```
fo = open("foo.txt", "r")  
line = fo.readline()  
print "Read Line: %s" % (line)  
line = fo.readline(5)  
print "Read Line: %s" % (line)
```

```
fo.close()
```

next() method:

File.next()

The method `next()` is used when a file is used as an iterator, typically in a loop, the `next()` method is called repeatedly.

This method returns the next input line, or raises *StopIteration when EOF is hit*.

Syntax

Following is the syntax for **next() method**:

```
fileObject.next();
```

Example

```
fo = open("foo.txt", "r")

for index in range(5):

    line = fo.next()

    print "Line No %d - %s" % (index, line)

fo.close()
```

File Positions

The ***tell()*** method tells you the current position within the file.

The ***seek(offset[, from])*** method changes the current file position.

The ***offset*** argument indicates the number of bytes to be moved.

The ***from*** argument specifies the reference position from where the bytes are to be moved.

If ***from is set to 0***, it means use the beginning of the file as the reference position **1 means use** the current position as the reference position; if it **is set to 2** then the end of the file would be taken as the reference position.

Example:

```
fo = open("foo.txt", "r+")

str = fo.read(10);
```

112 Python Programming for Beginners

```
print "Read String is : ", str
# Check current position
position = fo.tell()
print "Current file position : ", position
# Reposition pointer at the beginning once again
position = fo.seek(0, 0);
str = fo.read(10);
print "Again read String is : ", str
# Close opened file
fo.close()
```

Renaming and Deleting Files:

Python os module provides methods that help you perform file-processing operations, such as renaming and deleting files.

The rename() Method:

The *rename()* method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

Example

```
import os
# Rename a file from test1.txt to test2.txt
os.rename( "test1.txt", "test2.txt" )
```

The remove() Method:

You can use the *remove()* method to delete files by supplying the name of the file to be deleted as the argument.

Syntax

```
os.remove(file_name)
```

Example

```
import os

# Delete file test2.txt

os.remove("text2.txt")
```

Directories in Python

The `os` module has several methods that help you create, remove, and change directories. You can use the *`mkdir()` method of the `os` module to create directories in the current directory.*

You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

Example:

```
import os

# Create a directory "test"

os.mkdir("test")
```

Exercise:

- Q.1 Explain the need of creation of file and write data into it.
- Q.2 Explain the `open()` function and access modes of file in detail.
- Q.3 Explain the `close()` method and how to close file automatically without using the `close()` method.
- Q.4 Explain `write()`, `read()`, `readline()`, `next()`, `tell()` and `seek()` method in detail.
- Q.5 Explain `os` module in detail.

Chapter 7

ERRORS AND EXCEPTIONS

- ✓ *Syntax Errors, Exceptions*
- ✓ *Handling Exceptions*
- ✓ *Exception Examples*
- ✓ *Raising Exceptions*
- ✓ *User-defined Exceptions*
- ✓ *Defining Clean-up Actions(try - finally) and Predefined Clean-up Actions*
- ✓ *Exercise 7*

7.1 Syntax Errors, Exceptions:

There are two distinguishable kinds of **errors**: *syntax errors and exceptions*

Syntax Errors:

Syntax errors, also known as parsing errors.

```
>>> while True print 'Hello world'
```

```
File "<stdin>", line 1
```

```
while True print 'Hello world'
```

```
^
```

SyntaxError: invalid syntax

The parser repeats the offending line and displays a little 'arrow' pointing at the earliest point in the line where the error was detected.

The error is caused by the token *preceding the arrow*:

*In the example, the error is detected at the keyword **print**, since a colon (':') is missing before it.*

Exception:

Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called **exceptions**.

Exception can be said to be any abnormal condition in a program resulting in the disruption in the flow of the program.

Common Exceptions:

ZeroDivisionError:

Occurs when a number is divided by zero.

NameError:

It occurs when a name is not found. It may be local or global.

IndentationError:

If incorrect indentation is given.

IOError:

It occurs when Input Output operation fails.

EOFError:

It occurs when end of the file is reached and yet operations are being performed.

Examples:

```
>>> 10 * (1/0)
```

ZeroDivisionError: integer division or modulo by zero

```
>>> 4 + spam*3
```

NameError: name 'spam' is not defined

```
>>> '2' + 2
```

TypeError: cannot concatenate 'str' and 'int' objects

```
>>> from fractions import Fraction
```

```
>>> frac = Fraction(1, 2)
```

```
>>> print(frac.numerator)
```

```
1
```

```
>>> print(frac.numertor)
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

AttributeError: 'Fraction' object has no attribute '**numertor**'

```
>>> from fraction import Fraction
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

ImportError: No module named '**fraction**'

7.2 Handling Exceptions:

The suspicious code can be handled by using the **try block**.

Enclose the code which raises an exception inside the try block.

The try block is followed by the **except** statement.

It is then further followed by statements which are executed during exception and in case if exception does not occur.

Syntax:

try:

malicious code

except Exception1:

execute code

except Exception2:

execute code

....

....

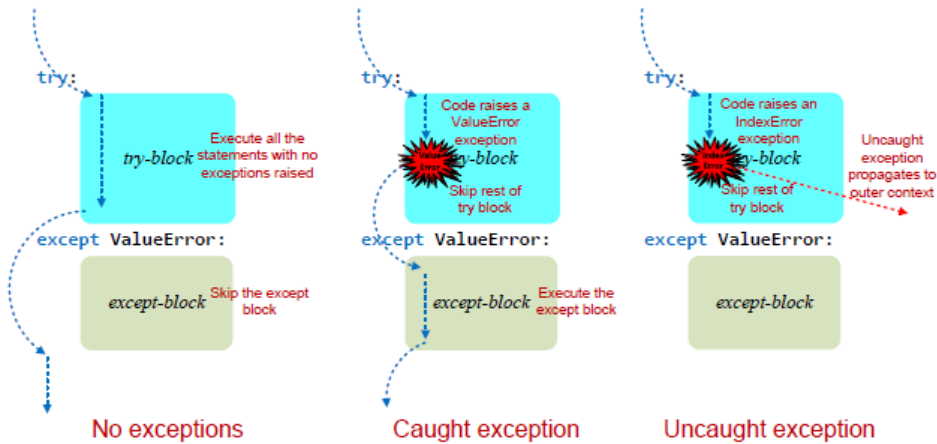
except ExceptionN:

execute code

else:

In case of no exception, execute the else block code.

Figure 12.1 The possible program execution flows through a try/except statement.

**Example:1****try:**`a=10/0``print a`**except** `ArithmeticError :``print "This statement is raising an exception"`**Output:**`>>>``This statement is raising an exception``>>>`**# Example:2**

Assuming we want to ask the user to enter an integer number. If we use a `raw_input()`, the input will be a string, which we have to cast into an integer.

If the input has not been a valid integer, we will generate (raise) a `ValueError`.

Follow this by using interactive session:

```
>>> n = int(raw_input("Please enter a number: "))
```

With the aid of exception handling, we can write robust code for reading an integer from input:

while True:

try:

n = raw_input("Please enter an integer: ")

n = int(n)

break

except ValueError:

print("No valid integer! Please try again...")

7.3 Python Exception(Except with No Exception) Example:

Syntax:

try:

code

except:

code to be executed in case exception occurs.

else:

code to be executed in case exception does not occur.

Example:

try:

a=10/0

print a

except :

print "Arithmetic Exception"

else:

print "successfully done"

Output:

```
>>>
Arithmetic Exception
>>>
```

Declaring Multiple Exception in Python:**Syntax:****try:**

code

except Exception1,Exception2,Exception3,...,ExceptionN

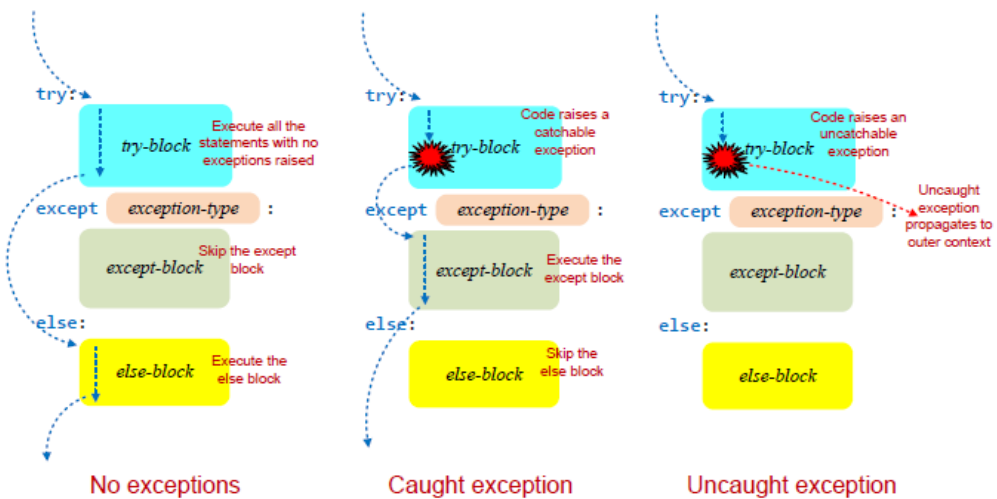
execute this code in case any Exception of these occur.

else:

execute code in case no exception occurred.

Handling Exceptions(else block):

Figure 12.6 The possible program execution flows through a try/else statement.



Example:1

```

try:
    a=10/0
    print a
except :ArithmeticError,StandardError
    print "Arithmetic Exception"
else:
    print "successfully done"

```

Output:

Arithmetic Exception

Example:2

It shows a try clause, in which we open a file for reading, read a line from this file and convert this line into an integer. There are at least two possible exceptions:

an IOError

ValueError

Just in case we have an additional unnamed except clause for an unexpected error:

```

import sys

try:
    f = open('integers.txt')
    s = f.readline()
    i = int(s.strip())
except (IOError, ValueError):
    print "I/O error or Value error occurred"
except:
    print "Unexpected error occured"

```

```

import random

for i in range(10):    # Loop 10 times
    print('Beginning of loop iteration', i)
    try:
        r = random.randint(1, 3)    # r is pseudorandomly 1, 2, or 3
        if r == 1:
            print(int('Fred'))    # Try to convert a non-integer
        elif r == 2:
            [[]][2] = 5    # Try to assign to a nonexistent index of the empty list
        else:
            print(3/0)    # Try to divide by zero
    except ValueError:
        print('Cannot convert integer')
    except IndexError:
        print('List index is out of range')
    except ZeroDivisionError:
        print('Division by zero not allowed')

    print('End of loop iteration', i)

```

7.4 Raise an Exception:

You can explicitly throw an exception in Python using raise statement.

The raise statement will cause an exception to occur and thus execution control will stop in case it is not handled.

Syntax:

```
raise Exception_class,<value>
```

Example

```
try:
```

```
    a=10
```

```
    print a
```

```
raise NameError("Hello")
```

```
except NameError as e:
```

```
    print "An exception occurred"
```

```
    print e
```

Output:

10

An exception occurred

Hello

Assertions in Python:

An expression is tested, and if the result comes up false, an exception is raised.

Assertions are carried out by the `assert` statement.

Programmers often place assertions at the start of a function to check for valid input, and after a function call to check for valid output.

When it encounters an `assert` statement, Python evaluates the accompanying expression, which is hopefully true.

If the expression is false, Python raises an *AssertionError* exception.

The syntax for `assert` is –

```
assert Expression[, Arguments]
```

If the assertion fails, Python uses `ArgumentExpression` as the argument for the `AssertionError`.

Example:

Function that converts a temperature from **degrees Kelvin to degrees Fahrenheit**. Since **zero degrees Kelvin** is as cold as it gets, the function bails out if it **sees a negative temperature** –

```
def KelvinToFahrenheit(Temperature):
    assert (Temperature >= 0), "Colder than absolute zero!"
    return ((Temperature-273)*1.8)+32
print KelvinToFahrenheit(273)
print int(KelvinToFahrenheit(505.78))
print KelvinToFahrenheit(-5)
```

7.5 User-defined Exceptions:

Programs may name their own exceptions by creating a new exception.

Exceptions should typically be derived from the `Exception` class, either directly or indirectly.

Example:

```

class MyError(Exception):
    def __init__(self, value):
        self.value = value

try:
    raise MyError(2*2)

except MyError as e:
    print 'My exception occurred, value:', e.value

My exception occurred, value: 4

```

7.6 Clean-Up Actions (Try ... Finally):

The **try** statement had always been paired with **except** clauses.

But there is another way to use it as well.

The **try** statement can be followed by a **finally clause**. Finally clauses are called **clean-up or termination clauses**, because they must be executed under all circumstances, i.e. a “finally” clause is always executed regardless if an exception occurred in a try block or not.

Syntax:

try:

Code

finally:

code which is must to be executed.

Example1:

try:

```

x = float(raw_input("Your number: "))
inverse = 1.0 / x

```

finally:

```
print("There may or may not have been an exception.")
```

```
print "The inverse: ", inverse
```

The **try** statement had always been paired with **except** clauses.

But there is another way to use it as well.

The **try** statement can be followed by a **finally clause**. Finally clauses are called **clean-up or termination clauses**, because they must be executed under all circumstances, i.e. a “finally” clause is always executed regardless if an exception occurred in a try block or not.

Syntax:**try:**

```
Code
```

finally:

```
code which is must to be executed.
```

Example2:**try:**

```
x = float(raw_input("Your number: "))
```

```
inverse = 1.0 / x
```

finally:

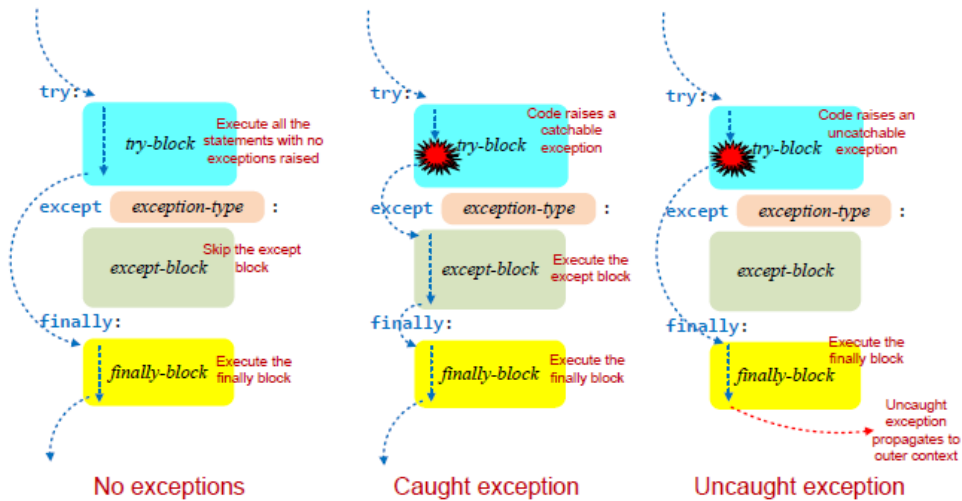
```
print("There may or may not have been an exception.")
```

```
print "The inverse: ", inverse
```

COMBINING TRY, EXCEPT AND FINALLY:

“finally” and “except” can be used together for the same try block.

Figure 12.7 The possible program execution flows through a try/except/finally statement.

**Example:****try:**

```
x = float(raw_input("Your number: "))
```

```
inverse = 1.0 / x
```

except ValueError:

```
print "You should have given either an int or a float"
```

except ZeroDivisionError:

```
print "Infinity"
```

finally:

```
print("There may or may not have been an exception.")
```

```
print "The inverse: ", inverse
```

Predefined Clean-up Actions

Some objects define standard clean-up actions to be undertaken when the object is no longer needed, regardless of whether or not the operation using the object succeeded or failed.

Example:

which tries to open a file and print its contents to the screen.

```
for line in open("myfile.txt"):
```

```
    print line,
```

The problem with this code is that it leaves the file open for an indeterminate amount of time after the code has finished executing.

This is not an issue in simple scripts, but can be a problem for larger applications. The `with` statement allows objects like files to be used in a way that ensures they are always cleaned up promptly and correctly.

```
with open("myfile.txt") as f:
```

```
    for line in f:
```

```
        print line,
```

After the statement is executed, **the file *f* is always closed**, even if a problem was encountered while processing the lines.

Exercise:

- Q.1 Explain the difference between error and exception in detail.
- Q.2 What is Exception? Explain the common exception `ZeroDivisionError`, `NameError`, `TypeError` and `ValueError` in detail with examples.
- Q.3 Explain the concept handling of exception with examples.
- Q.4 Explain the concept raising of exception with examples.
- Q.5 Write a Python code to create user defined exception.
- Q.6 Explain clean up actions and predefined clean up actions in detail with examples.

Chapter 8

INTRODUCTION TO OBJECT ORIENTED CONCEPTS IN PYTHON

- ☑ *Object Oriented concepts*
- ☑ *Objects, Python Scopes and Namespaces*
- ☑ *Classes, Class Objects, Instance Objects, Method Objects, Class and Instance Variables.*
- ☑ *Inheritance*
- ☑ *Exercise 8*

8.1 Object Oriented concepts:

Python is an object-oriented programming language. It allows us to develop applications using Object Oriented approach. In Python, we can easily create and use classes and objects.

Major principles of object-oriented programming system are given below

Object

Class

Method

Inheritance

Polymorphism

Data Abstraction

Encapsulation

Object:

Object is an entity that has state and behavior. It may be anything. It may be physical and logical. For example: mouse, keyboard, chair, table, pen etc.

Everything in Python is an object, and almost everything has attributes and methods. All functions have a built-in attribute `__doc__`, which returns the doc string defined in the function source code.

Class

Class can be defined as a collection of objects. It is a logical entity that has some specific attributes and methods. For example: if you have an employee class then it should contain an attribute and method i.e. an email id, name, age, salary etc.

Syntax:

class ClassName:

<statement-1>

.

.

.

<statement-N>

Method

Method is a function that is associated with an object. In Python, method is not unique to class instances. Any object type can have methods.

Inheritance

Inheritance is a feature of object-oriented programming. It specifies that one object acquires all the properties and behaviors of the parent object. By using inheritance, you can define a new class with a little or no changes to the existing class. The new class is known as derived class or child class and from which it inherits the properties is called base class or parent class.

It provides re-usability of the code.

Polymorphism

Polymorphism is made by two words “poly” and “morphs”. Poly means many and Morphs means form, shape. It defines that one task can be performed in different ways. For example: You have a class animal and all animals talk. But they talk differently. Here, the “talk” behavior is polymorphic in the sense and totally depends on the animal. So, the abstract “animal” concept does not actually “talk”, but specific animals (like dogs and cats) have a concrete implementation of the action “talk”.

Encapsulation

Encapsulation is also the feature of object-oriented programming. It is used to restrict access to methods and variables. In encapsulation, code and data are wrapped together within a single unit from being modified by accident.

Data Abstraction

Data abstraction and encapsulation, both are often used as synonyms. Both are nearly synonyms because data abstraction is achieved through encapsulation.

Abstraction is used to hide internal details and show only functionalities. Abstracting something means to give names to things, so that the name captures the core of what a function or a whole program does.

8.3 Classes, Class Objects, Instance Objects, Method Objects, Class and Instance Variables: Class and Instance Variables in Python

There are **two main types**:

class variables: which have the same value across all class instances (i.e. static variables),

instance variables: which have different values for each object instance.

Syntax

class ClassName:

self.instance_variable = value *#value specific to instance*

class_variable = value *#value shared across all class instances*

#accessing instance variable

class_instance = ClassName()

class_instance.instance_variable

#accessing class variable

ClassName.class_variable

Example

class Car:

wheels = 4 *# class variable*

def __init__(self, make):

self.make = make *#instance variable*

newCar = Car("Honda")

print ("My new car is a {}".format(newCar.make))

print ("My car, like all cars, has {%d} wheels".format(Car.wheels))

Class and Instance methods in Python

Methods in Python

Methods are functions declared within a class.

There are three types:

instance, class, and static.

Instance methods have different implementations across all instances.

They are called in relation to the current instance.

Class methods have the same implementation across all instances. They are called in relation to the instance passed in.

Static methods have the same implementation across all instances. They are not tied to a class instance.

Class and Instance methods in Python

Syntax

```
class ClassName(object):
    #instance methods always take self as the first parameter
    def instance_method(self, passed_params):
        #code to execute

    #the constructor is an instance method
    def __init__(self, passed_params):
        #assignments

    #class methods always take a class instance as the first parameter
    @classmethod
    def class_method(cls, passed_params):
        #code to execute

    @staticmethod
    def static_method(passed_params):
        #code to execute

class_instance = ClassName()
#calling instance and class methods
class_instance.instance_method()
#calling static methods
ClassName.static_method()
```

Example

```
class Dog(object):
    #class attribute
    legs = 4

    #instance methods
    def __init__(self, breed):
        self.breed = breed

    def get_breed(self):
        return self.breed

    @classmethod
    def get_legs(cls):
        return cls.legs

    @staticmethod
    def make_noise():
        return "*bark*!"

new_dog = Dog("Golden Retriever")
print ("I got a new dog. It is a {}".format(new_dog.get_breed()))
print ("It goes {}".format(Dog.make_noise()))
```

8.4 Inheritance:

What is Inheritance?

Inheritance is a feature of Object Oriented Programming. It is used to specify that one class will get most or all of its features from its parent class. It is a very powerful feature which facilitates users to create a new class with a few or more modifications to an existing class. The new class is called child class or derived class and the main class from which it inherits the properties is called base class or parent class.

The child class or derived class inherits the features from the parent class, adding new features to it. It facilitates **re-usability of code**.

Python Multilevel Inheritance:

Multilevel inheritance is also possible in Python like other Object Oriented programming languages. We can inherit a derived class from another derived class; this process is known as multilevel inheritance. In Python, multilevel inheritance can be done at any depth.

Python Multilevel Inheritance:

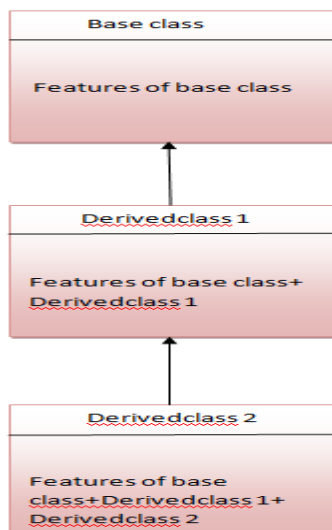
Multilevel inheritance is also possible in Python like other Object Oriented programming languages. We can inherit a derived class from another derived class; this process is known as multilevel inheritance. In Python, multilevel inheritance can be done at any depth.

```
# Multilevel Inheritance Example
class Employee: # Base class
    raise_amt=1.04
    def __init__(self,first,last,pay):
        self.first=first
        self.last=last
        self.email=first+'.'+last+'@bollywood.com'
        self.pay=pay

class Developer (Employee): # Derived class 1
    def __init__(self,prog_lang):
        Employee.__init__(self,first,last,pay)
        self.prog_lang=prog_lang

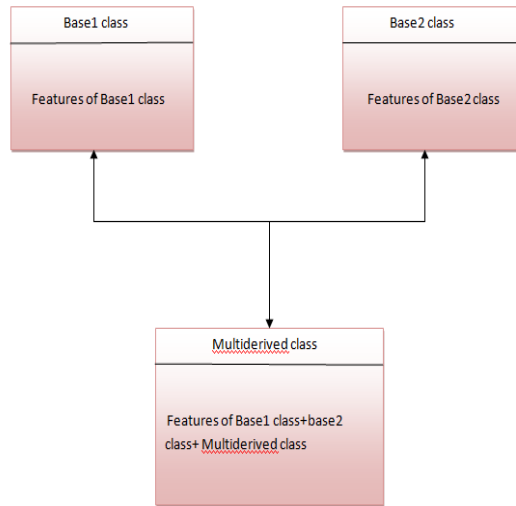
class Manager (Developer): # Manager class is derived class 2
    def __init__(self,first,last,pay,prog_lang,time):
        Developer.__init__(self,first,last,pay,prog_lang)
        self.time=time

mgr1=Manager('akib','Bagwan',25000,'python',10)
print(mgr1.pay)
print(mgr1.time)
print (mgr1.prog_lang)
```



Python Multiple Inheritance:

Python supports multiple inheritance too. It allows us to inherit multiple parent classes. We can derive a child class from more than one base (parent) classes.



```
# Multiple Inheritance Example
class Employee: # Base class 1
    raise_amt=1.04
    def __init__(self,first,last,pay):
        self.first=first
        self.last=last
        self.email=first+'.'+last+'@bollywood.com'
        self.pay=pay

class Developer(): # Base class 2
    def __init__(self,prog_lang):
        self.prog_lang=prog_lang

class Manager(Developer,Employee): # Manager class is derived class
    def __init__(self,first,last,pay,prog_lang,time):
        Employee.__init__(self,first,last,pay)
        Developer.__init__(self,prog_lang)
        self.time=time

mgr1=Manager('akib','Bagwan',25000,'python',10)
print(mgr1.pay)
print(mgr1.time)
print(mgr1.prog_lang)
```

Exercise:

- Q.1 Explain the object oriented concept in details.
- Q.2 Explain the instance variable and class variable with examples.
- Q.3 Explain the instance method, class method and static method with examples.
- Q.4 Write a python code to create animal class and in it create one instance variable, class variable, instance method and class method and access it through object and class names.
- Q.5 Explain multiple and multilevel inheritance with examples.

